

VERIQTA

VOLUME

2

ADVANCED DEVOPS INTERVIEW QUESTIONS

VOLUME 2: KUBERNETES, CONTAINERS & CLOUD NATIVE



KUBERNETES MASTERY

Production-grade K8s Questions



CONTAINERS & RUNTIME

Deep dive into containers & CRI



CLOUD NATIVE ARCHITECTURE

Design scalable cloud native systems



GITOPS & AUTOMATION

Argo CD, Flux & Automation



TROUBLESHOOTING EXPERTISE

Debug like a Senior Engineer



SECURITY & HARDENING

Secure clusters, best practices



PRODUCTION READINESS

What senior engineers look for

Real-world. Production-focused.
Senior Engineer Level.



THINK LIKE A SENIOR.
BUILD LIKE A LEADER.

200+ ADVANCED INTERVIEW QUESTIONS

- ✓ Real Production Scenarios
- ✓ Senior Engineer Mindset
- ✓ System Design & Deep Dives
- ✓ End-to-End Troubleshooting
- ✓ Kubernetes & Cloud Native Best Practices



SENIOR LEVEL
DEPTH



PRODUCTION
FOCUSED



REAL-WORLD
SCENARIOS



INTERVIEW
READY



CAREER
ACCELERATOR

VERIQTA



@veriqta_



Veriqta



t.me/veriqta



veriqta.com

SENIOR KUBERNETES ARCHITECTURE

Real interview questions. Real production architectures.

TOPICS COVERED:

✓ Control Plane ✓ Worker Nodes ✓ API Server Workflow ✓ etcd Reliability ✓ Scheduler ✓ Controller Manager ✓ Production Design

① Q: Explain the Kubernetes architecture at a high level.

ANS: • Kubernetes follows a master-worker architecture.
• Control Plane manages the cluster and makes global decisions.
• Worker Nodes run containerized applications.
• All communication is through the Kubernetes API Server.
• etcd is the source of truth for all cluster state.

INTERVIEWER LOOKS FOR:

- Correct components
- Clear responsibilities
- Understanding of flow
- Production awareness
- Ability to explain simply

② Q: What are the core components of the Kubernetes Control Plane?

ANS: • API Server – Frontend for the cluster.
• etcd – Consistent and highly available key-value store.
• Scheduler – Decides which node will run new pods.
• Controller Manager – Runs core control loops.
• Cloud Controller Manager – Integrates with cloud provider.
• (Optional) Aggregation Layer – Extends the API.

INTERVIEWER LOOKS FOR:

- Component knowledge
- Responsibilities
- etcd importance
- Real production insight
- Understanding HA needs

③ Q: How does the Kubernetes API Server work?

ANS: • All clients (kubectl, controllers, nodes) communicate with API Server using REST over HTTPS.
• API Server authenticates, authorizes, validates requests.
• It reads/writes cluster state in etcd.
• It notifies components through watches.
• It is the most critical component in the control plane.

INTERVIEWER LOOKS FOR:

- API flow understanding
- AuthN, AuthZ, Admission
- etcd interaction
- Watch mechanism
- Critical thinking

④ Q: How does the Scheduler make decisions?

ANS: • Watches for new pods without a node assigned.
• Filters nodes based on resource availability, constraints, taints/tolerations, node selectors, etc.
• Scores remaining nodes using priorities.
• Picks the highest scoring node and binds the pod.

INTERVIEWER LOOKS FOR:

- Scheduling cycle
- Filtering vs Scoring
- Constraints handling
- Extensibility knowledge
- Real-world scenarios

⑤ Q: What does the Controller Manager do?

ANS: • Runs controller loops that keep the cluster in the desired state.
• Key controllers: Deployment, ReplicaSet, Node, Endpoints, Namespace, ServiceAccount, etc.
• Continuously watches the state via API Server and reconciles differences.

INTERVIEWER LOOKS FOR:

- Reconciliation concept
- Controller examples
- Self-healing understanding
- Production debugging
- Ownership mindset

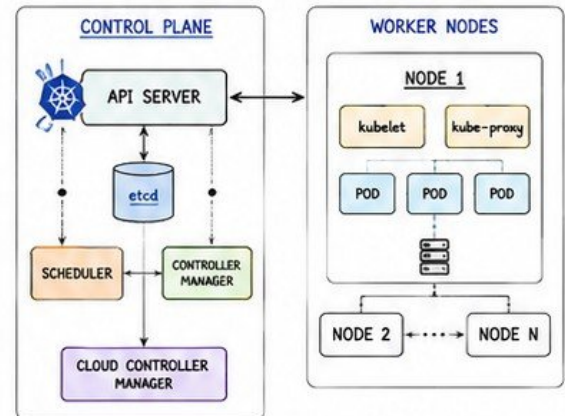
⑥ Q: How do you design a production-ready Kubernetes cluster?

ANS: • Use highly available control plane (≥ 3 replicas).
• Use external etcd or stacked etcd with quorum (odd number).
• Distribute control plane across multiple AZs.
• Separate worker nodes across AZs.
• Use PodDisruptionBudgets, ResourceQuotas, LimitRanges.
• Monitor everything. Automate backups and upgrades.
• Plan for failure: HA, DR, rollback, and incident response.

INTERVIEWER LOOKS FOR:

- HA design
- Reliability mindset
- Multi-AZ awareness
- Backup & restore
- Operational excellence

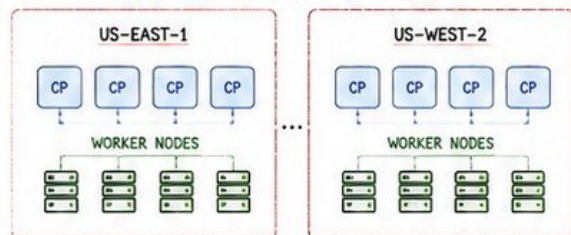
KUBERNETES ARCHITECTURE OVERVIEW



COMPONENT RESPONSIBILITIES

API Server	Exposes the Kubernetes API. Handles all requests and updates to cluster state.
etcd	Distributed key-value store. Source of truth for the entire cluster.
Scheduler	Assigns pods to the best suitable node.
Controller Manager	Runs controllers to maintain desired state.
Cloud Controller Manager	Integrates Kubernetes with cloud provider APIs (Load Balancers, Volumes, Routes).
kubelet (Node Agent)	Ensures containers are running in a Pod.
kube-proxy	Maintains network rules on nodes for Service connectivity.

TYPICAL PRODUCTION DEPLOYMENT



CP = Control Plane

✓ Multi-AZ ✓ Highly Available ✓ Fault Tolerant



KEY TAKEAWAY

Kubernetes is a distributed system. The control plane makes decisions, and the worker nodes run your workloads. Design for failure, automate everything, and always assume something will break.



PRO TIP

Always think in terms of: 1) Availability 2) Scalability 3) Security 4) Observability 5) Operability
A good Kubernetes engineer designs for today's load and tomorrow's failure.

POD LIFECYCLE & SCHEDULING

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Pod Lifecycle
- ✓ Init Containers
- ✓ Scheduling Constraints
- ✓ Node Selectors
- ✓ Taints & Tolerations
- ✓ Affinity & Anti-Affinity
- ✓ Production Scheduling Failures

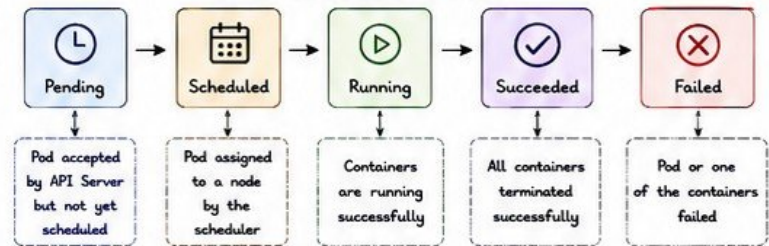
1 Q: What is the Pod lifecycle?

ANS: A Pod moves through phases as it is created, scheduled, starts containers, runs, and eventually completes or fails.

INTERVIEWER LOOKS FOR:

- Understanding of phases
- Transition awareness
- Production insight
- Troubleshooting mindset

POD LIFECYCLE PHASES



Other states you should know: Unknown (node lost contact), Terminating (graceful shutdown)

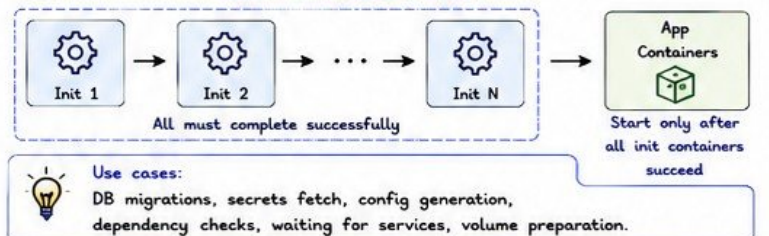
2 Q: What are Init Containers and when do you use them?

ANS: Init containers run before app containers start. Each must complete successfully before the next one runs.

INTERVIEWER LOOKS FOR:

- Purpose clarity
- Execution order
- Failure behavior
- Real-world usage

INIT CONTAINER FLOW



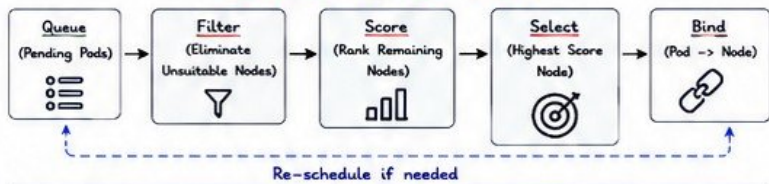
3 Q: What factors does the scheduler consider when placing a Pod?

ANS: Scheduler looks at resources, constraints, policies, affinity/anti-affinity, taints/tolerations, and other rules.

INTERVIEWER LOOKS FOR:

- Scheduling cycle
- Filters vs Scores
- Constraints handling
- Extensibility knowledge

SCHEDULER DECISION FLOW



4 Q: What is a Node Selector?

ANS: Node Selector is a simple way to constrain Pods to nodes with matching labels.

INTERVIEWER LOOKS FOR:

- Label based selection
- Simple vs advanced
- Use case awareness

EXAMPLE spec:

```

nodeSelector:
  disktype: ssd
  environment: production
  
```

MATCHES NODES WITH LABELS:

```

disktype=ssd
environment=production
  
```

5 Q: What are Taints and Tolerations?

ANS: Taints prevent Pods from being scheduled on nodes unless the Pod has a matching toleration.

INTERVIEWER LOOKS FOR:

- Taint effect types
- Why used
- Toleration match
- Common mistakes

TAINTS (ON NODES)

NoSchedule Pod cannot be scheduled

PreferNoSchedule Try to avoid

NoExecute Evict existing pods that don't tolerate

TOLERATIONS (ON PODS)

tolerations:

- key: "dedicated"
- operator: "Equal"
- value: "gpu"
- effect: "NoSchedule"

6 Q: What is the difference between Affinity and Anti-Affinity?

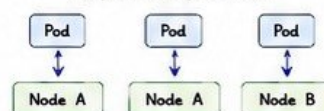
ANS: Affinity attracts Pods to nodes or Pods. Anti-Affinity prevents Pods from being placed together.

INTERVIEWER LOOKS FOR:

- Pod vs Node affinity
- Soft vs Hard rules
- Topology awareness
- Real-world scenarios

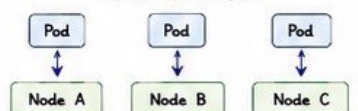
POD AFFINITY (Attract)

Keep related pods close



POD ANTI-AFFINITY (Avoid)

Spread pods apart



Use topologyKey: `kubernetes.io/hostname` for zone-level spreading.

7 Q: What are common reasons Pods remain in Pending state?

ANS: Insufficient resources, node selectors don't match, taints without tolerations, volume issues, quota limits, or scheduling constraints.

INTERVIEWER LOOKS FOR:

- Root cause analysis
- Debugging steps
- Experience depth
- Preventive mindset

COMMON PENDING REASONS & HOW TO INVESTIGATE

REASON	SYMPTOMS	HOW TO INVESTIGATE
Insufficient Resources	Events: Insufficient cpu/memory	<code>kubectl describe pod, top nodes</code>
Node Selector Mismatch	O/Nodes are available	Check labels: <code>kubectl get nodes --show-labels</code>
Taints Without Toleration	didn't tolerate taint	<code>kubectl describe node</code>
Volume / PVC Issues	PVC pending / attach errors	<code>kubectl describe pvc</code>
Quota / Limit Ranges	exceeds quota	<code>kubectl describe quota</code>
Scheduling Constraints	Affinity rules not satisfied	Review affinity/anti-affinity rules



PRO TIP

When a Pod is Pending, always check: 1) Events 2) Resources 3) Constraints 4) Taints 5) Volumes
Tools: `kubectl describe pod <pod>` | `kubectl get events --sort-by=.lastTimestamp`

KUBERNETES NETWORKING

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Pod-to-Pod Communication
- ✓ Services
- ✓ kube-proxy
- ✓ CNI Plugins
- ✓ ClusterIP vs NodePort vs LoadBalancer
- ✓ Ingress Controllers
- ✓ Network Troubleshooting

1 Q: How do Pods communicate with each other?

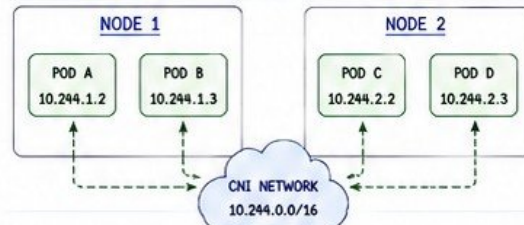
ANS:

- All Pods get an IP address from the cluster network (CNI plugin).
- Pods can communicate with each other directly using IP addresses.
- No NAT is performed between Pods.
- Communication works across all nodes.
- This is flat, routable network inside the cluster.

INTERVIEWER LOOKS FOR:

- Understanding of CNI
- Flat vs non-flat networks
- Routing between nodes
- Network isolation
- Real-world knowledge

POD-TO-POD COMMUNICATION



KEY IDEA

Every Pod can reach every other Pod, no matter which node it is on. (Flat network)

2 Q: What is a Service in Kubernetes?

ANS:

- A Service is a stable network endpoint for a set of Pods.
- It provides load balancing and service discovery.
- It abstracts Pod IPs which are ephemeral.
- Services enable communication within and outside the cluster.

INTERVIEWER LOOKS FOR:

- Service abstraction
- Load balancing
- Service discovery
- Types of Services
- Use cases

SERVICE ABSTRACTION



REMEMBER

Clients talk to the Service IP. Service forwards traffic to healthy Pods (Endpoints).

3 Q: What does kube-proxy do?

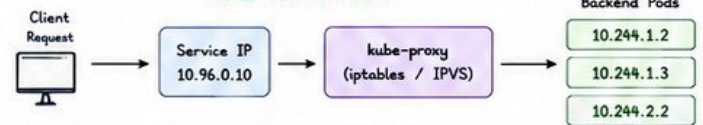
ANS:

- Runs on every node.
- Implements Service abstraction.
- Maintains network rules (iptables/IPVS) to forward traffic to the correct Pods.
- Ensures load balancing based on the Service type.

INTERVIEWER LOOKS FOR:

- Purpose of kube-proxy
- iptables vs IPVS
- Service routing
- Performance impact
- Real production issues

KUBE-PROXY FLOW



kube-proxy programs rules so that traffic to the Service IP is forwarded to one of the healthy Pod IPs.

4 Q: What are CNI plugins?

ANS:

- CNI (Container Network Interface) plugins provide network connectivity for Pods.
- They assign IP addresses and setup routing.
- Examples: Calico, Cilium, Flannel, Weave, Amazon VPC CNI, Azure CNI.

INTERVIEWER LOOKS FOR:

- CNI purpose
- Popular plugins
- Overlay vs Underlay
- Network policies
- Production experience

COMMON CNI PLUGINS



SERVICE TYPES: CLUSTERIP vs NODEPORT vs LOADBALANCER

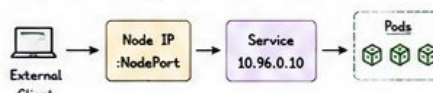
CLUSTERIP (DEFAULT)

- Accessible only within the cluster.
- Has a virtual IP from the cluster IP range.
- Ideal for internal communication.



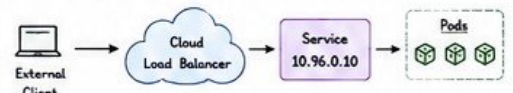
NODEPORT

- Exposes Service on a static port on each Node.
- Accessible from outside using <NodeIP>:<NodePort>.
- Useful for testing / small environments.



LOADBALANCER

- Cloud provider provisions a Load Balancer.
- Exposes Service externally with a public IP/DNS.
- Best for production workloads.



5 Q: What is an Ingress Controller?

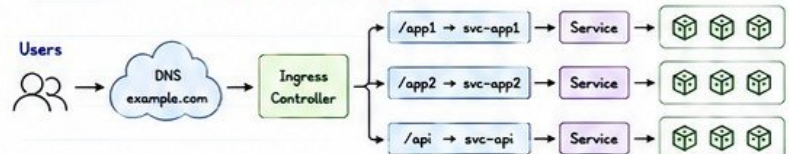
ANS:

- Manages external HTTP/HTTPS access to Services.
- Provides routing (host/path based).
- TLS termination, auth, rate limiting, WAF.
- Examples: NGINX Ingress, Traefik, HAProxy, Istio Ingress Gateway.

INTERVIEWER LOOKS FOR:

- Role of Ingress
- Ingress vs LoadBalancer
- Annotations
- TLS handling
- Real-world usage

INGRESS ARCHITECTURE



6 Q: How do you troubleshoot networking issues?

ANS:

- Check Pod IP and connectivity.
- Check DNS resolution.
- Verify Services and Endpoints.
- Check Network Policies.
- Inspect CNI logs and node networking.

INTERVIEWER LOOKS FOR:

- Troubleshooting mindset
- Tools knowledge
- Layered approach
- Hypothesis driven
- Production experience

NETWORK TROUBLESHOOTING CHECKLIST

1 Pod has IP?	<code>kubectl get pod -o wide</code>	5 Check Network Policies	<code>kubectl get networkpolicies -A</code>
2 Can Pod ping another Pod?	<code>kubectl exec -it <pod> -- ping <ip></code>	6 Check CNI logs on node	<code>journalctl -u <cni-agent></code>
3 DNS working?	<code>kubectl exec -it <pod> -- nslookup <svc></code>	7 Check kube-proxy logs	<code>kubectl logs -n kube-system -l k8s-app=kube-proxy</code>
4 Service has Endpoints?	<code>kubectl get endpoints <svc></code>	8 Verify routing / iptables	<code>ip route iptables -t nat -L -n</code>



PRO TIP

Networking is about understanding the flow. Always draw the flow before debugging.



Golden Rule: Always test from the failing component, not from your laptop.

PRODUCTION KUBERNETES FAILURE SCENARIOS

Real interview questions. Real production incidents.

TOPICS COVERED:

- ✓ CrashLoopBackOff
- ✓ Evicted Pods
- ✓ ImagePullBackOff
- ✓ NotReady Nodes
- ✓ Pending Pods
- ✓ Production Debugging Workflow
- ✓ ContainerCreating

1 Q: What causes CrashLoopBackOff and how do you fix it?

ANS:

- Container starts, crashes, and kubelet restarts it repeatedly.
- Check logs, exit code, config, dependencies.
- Fix the issue and redeploy.
- Use probes and resource limits to prevent future loops.

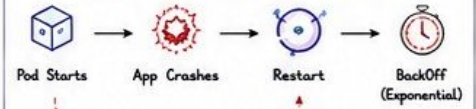
COMMON CAUSES:

- Application error / bad config
- Missing dependencies
- Port already in use
- OOM (memory)
- Readiness/Liveness probe failure

INTERVIEWER LOOKS FOR:

- Root cause analysis
- Logs and events usage
- Debugging mindset
- Preventive thinking
- Production readiness

HOW CRASHLOOPBACKOFF HAPPENS



2 Q: What causes ImagePullBackOff and how do you resolve it?

ANS:

- Kubelet can't pull the container image.
- Check image name, registry, credentials, network and permissions.
- Fix and delete pod (it will retry).
- Use imagePullSecrets for private registries.

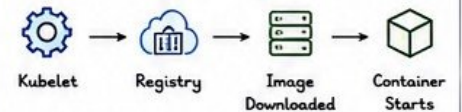
COMMON CAUSES:

- Wrong image name / tag
- Image doesn't exist
- Private registry auth failure
- Network / DNS issues
- Registry rate limits

INTERVIEWER LOOKS FOR:

- Understanding of image pull flow
- Auth & secret knowledge
- Network troubleshooting
- Security awareness

IMAGE PULL FLOW



If any step fails → ImagePullBackOff

3 Q: Why is a Pod in Pending state?

ANS:

- Scheduler can't find a suitable node.
- Check events: insufficient resources, taints, node selectors, affinity rules, PVC not bound, quota limits.
- Adjust constraints or add resources.

COMMON CAUSES:

- Insufficient CPU / Memory
- Taints not tolerated
- Node selector mismatch
- PVC not bound
- Pod/Namespaces quota exceeded

INTERVIEWER LOOKS FOR:

- Scheduling knowledge
- Event analysis
- Resource management
- Constraint handling

PENDING POD - COMMON REASONS



4 Q: What does ContainerCreating mean?

ANS:

- Kubelet is trying to create the container but something is blocking it.
- Check events, CNI, storage, permissions, image pull, or runtime issues.
- Fix the underlying issue.

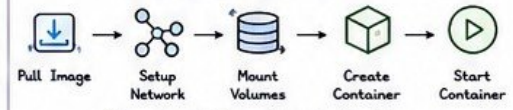
COMMON CAUSES:

- Image still pulling
- CNI plugin issue
- Volume mount failure
- Insufficient disk space
- Runtime / CRI errors

INTERVIEWER LOOKS FOR:

- Node-level debugging
- Volume & network knowledge
- Event interpretation
- System troubleshooting

CONTAINERCREATING FLOW



If any step fails → ContainerCreating

5 Q: Why are Pods getting Evicted?

ANS:

- Node is under pressure.
- Check events and resource usage.
- Evictions are done to protect the node.
- Fix resource pressure and tune requests/limits.

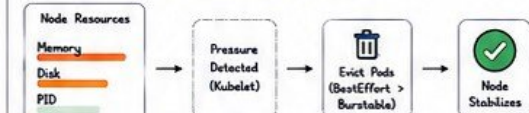
COMMON CAUSES:

- Memory pressure
- Disk pressure
- PID pressure
- Node pressure conditions
- kubelet eviction thresholds

INTERVIEWER LOOKS FOR:

- Resource management
- Node pressure knowledge
- Eviction thresholds
- Capacity planning

NODE PRESSURE & EVICTION



6 Q: What does NotReady node mean?

ANS:

- Kubelet is not reporting healthy status.
- Check kubelet, network, disk, container runtime, and resource usage.
- Node won't accept new Pods.
- Fix issue and wait for node to recover.

COMMON CAUSES:

- Kubelet down or unhealthy
- Network unreachable
- Disk full
- Container runtime down
- Cloud provider issues

INTERVIEWER LOOKS FOR:

- Node troubleshooting
- Infrastructure awareness
- System services knowledge
- Observability usage

NOTREADY NODE CHECKLIST



7 Q: Walk me through your production debugging workflow.

ANS:

- Gather information.
- Form hypothesis.
- Validate step by step.
- Mitigate or fix.
- Prevent recurrence.

KEY STEPS:

- 1 Identify the impact
- 2 Gather data (logs, events, metrics)
- 3 Form hypotheses
- 4 Test & validate
- 5 Mitigate / Fix
- 6 Root cause & document
- 7 Prevent future incidents

INTERVIEWER LOOKS FOR:

- Structured approach
- Thinking before acting
- Root cause mindset
- Communication
- Continuous improvement

PRODUCTION DEBUGGING WORKFLOW



Golden Rule:
Don't guess. Observe. Validate. Document. Improve.

QUICK DEBUGGING COMMANDS

Get Pod & Status

```
kubectl get pods -A
kubectl describe pod <pod> -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

Logs & Previous Logs

```
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> -c <container>
kubectl logs <pod> -n <ns> --previous
```

Node & Resource Info

```
kubectl get nodes
kubectl describe node <node>
kubectl top nodes
kubectl top pods -A
```

Image & Registry

```
kubectl describe pod <pod> -n <ns> | grep -i image
crictl images
crictl pull <image>
```

Networking

```
kubectl get svc -A
kubectl get endpoints -A
kubectl exec -it <pod> -- sh
nslookup <service>
```



PRO TIP

Read the events first. They tell you what Kubernetes is trying to do and where it is failing.

REMEMBER

- ✓ Verify Config
- ✓ Validate Assumptions
- ✓ Check Events
- ✓ Check Logs



INTERVIEWER WANTS TO SEE:

- Calm under pressure
- Clear communication
- Strong fundamentals
- System thinking
- Real production experience

CONTAINER RUNTIME & DOCKER

Real interview questions. Real production scenarios.

TOPICS COVERED:

✓ containerd vs CRI-O ✓ OCI Runtime ✓ Image Layers ✓ Runtime Failures ✓ Resource Isolation ✓ Namespaces & cgroups ✓ Runtime Troubleshooting

1 Q: What is the difference between containerd and CRI-O?

ANS:

- containerd is developed by CNCF and widely used.
- CRI-O is lightweight, written in Go, and designed for Kubernetes.
- Both implement the Kubernetes CRI (Container Runtime Interface).
- containerd has broader ecosystem support.

INTERVIEWER LOOKS FOR:

- Understanding of CRI
- Pros/cons comparison
- Kubernetes integration
- Production experience
- Trade-off thinking

CONTAINERD vs CRI-O

containerd

- CNCF graduated project
- Mature ecosystem
- Supports multiple runtimes (runc, gVisor, Kata)
- Used by Docker, Kubernetes, many platforms
- More features, larger footprint

CRI-O

- Lightweight, written in Go
- Minimal dependencies
- Optimized for Kubernetes
- Faster startup
- Smaller attack surface

VS

2 Q: What is an OCI Runtime?

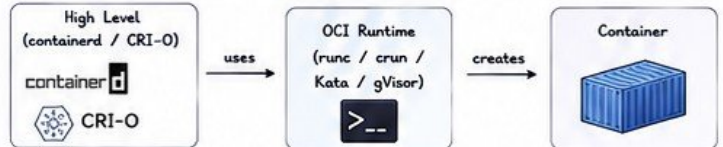
ANS:

- OCI Runtime is a low-level component that runs containers according to the OCI specification.
- Examples: runc, crun, Kata Containers, gVisor.
- It creates containers, manages namespaces, cgroups, and mounts.
- containerd/CRI-O use it to run containers.

INTERVIEWER LOOKS FOR:

- OCI standards knowledge
- Runtime responsibilities
- Examples
- Architecture awareness
- Low-level understanding

OCI RUNTIME ARCHITECTURE



3 Q: Explain Docker image layers.

ANS:

- Images are built in layers (read-only).
- Each layer represents an instruction in the Dockerfile.
- Layers are cached and reused.
- Final layer is the writable container layer.

INTERVIEWER LOOKS FOR:

- Layered filesystem
- Caching benefits
- Image optimization
- Storage efficiency
- Real-world awareness

DOCKER IMAGE LAYERS

Example Dockerfile

```

1 FROM ubuntu:22.04
2 RUN apt-get update
3 RUN apt-get install -y nginx
4 COPY ./app
5 CMD ["nginx", "-g", "daemon off;"]
  
```



Each layer is read-only and reused if unchanged

Container (Writable Layer)

4 Q: What are common container runtime failures?

ANS:

- Image pull failures (network, auth, registry).
- CrashLoopBackOff (app crashes).
- OOM kills (memory limits exceeded).
- Permission issues.
- Volume mount failures.

INTERVIEWER LOOKS FOR:

- Awareness of real issues
- Root cause thinking
- Debugging approach
- Preventive mindset
- Production experience

COMMON RUNTIME FAILURES



5 Q: How are resources isolated in containers?

ANS:

- CPU, memory, I/O, and network are isolated using cgroups and namespaces.
- Limits and requests control resource usage.
- Prevents one container from impacting others.

INTERVIEWER LOOKS FOR:

- Isolation mechanisms
- Resource controls
- cgroups understanding
- Production impact
- Best practices

RESOURCE ISOLATION (cgroups)

- CPU (cfs quota, shares)
- Memory (limit, reservation)
- I/O (blkio, throttling)
- PIDs (process limits)

NAMESPACE ISOLATION

- PID Namespace (process isolation)
- Network Namespace (network isolation)
- Mount Namespace (filesystem isolation)
- IPC Namespace (inter-process comms)

KEY IDEA

Containers share the host kernel but are isolated from each other using namespaces and cgroups.

6 Q: What is the role of namespaces in containers?

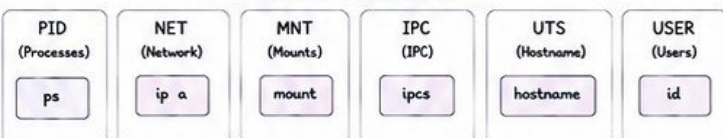
ANS:

- Provide isolated views of system resources.
- Types: PID, NET, MNT, IPC, UTS, USER.
- Each container gets its own isolated environment.
- Prevents containers from seeing each other's processes, network, or mounts.

INTERVIEWER LOOKS FOR:

- Namespace types
- Isolation concept
- Security awareness
- Real-world usage
- Deep understanding

TYPES OF NAMESPACES



7 Q: How do you troubleshoot container runtime issues?

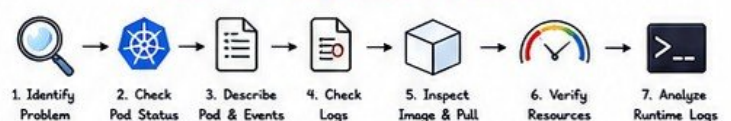
ANS:

- Check container status and events.
- Inspect logs and describe output.
- Verify image, registry, and credentials.
- Check resources, volume mounts, and network.
- Use runtime logs (containerd/cri-o) for deep issues.

INTERVIEWER LOOKS FOR:

- Systematic approach
- Right tools usage
- Root cause focus
- Logs & metrics
- Debugging mindset

RUNTIME TROUBLESHOOTING WORKFLOW



Check Status

```

kubectl get pods -A
kubectl describe pod <pod>
kubectl get events --sort-by=lastTimestamp
  
```

Logs

```

kubectl logs <pod>
kubectl logs <pod> -c <container>
kubectl logs <pod> --previous
  
```

Runtime (Node Level)

```

crictl ps -a
crictl images
crictl logs <container-id>
  
```

Resources

```

kubectl top pod <pod>
kubectl top node
kubectl describe node <node>
  
```

Image / Registry

```

docker pull <image>
docker images
docker login <registry>
  
```



PRO TIP

When debugging, always start from the highest level and go deeper.
Observe → Hypothesize → Validate → Fix → Prevent.



Golden Rule: Understand the layers:
App → Container → Runtime → OS → Hardware

PRODUCTION DEPLOYMENTS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Rolling Updates
- ✓ Blue/Green Deployments
- ✓ Canary Releases
- ✓ Progressive Delivery
- ✓ Feature Flags
- ✓ Rollback Strategy
- ✓ Zero Downtime Deployments

1 Q: What is Rolling Update in Kubernetes and how does it work?

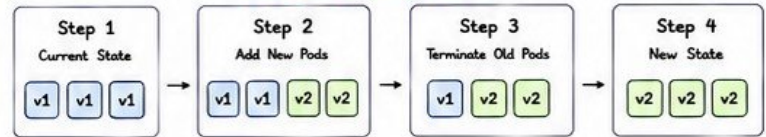
ANS:

- Incrementally replaces old Pods with new ones.
- Ensures availability during updates.
- Controlled by maxSurge and maxUnavailable.
- If issues occur, rollout can be paused or rolled back.

INTERVIEWER LOOKS FOR:

- Understanding of strategy
- k8s deployment concepts
- maxSurge / maxUnavailable
- Rollout control
- Production awareness

ROLLING UPDATE FLOW



Key: maxSurge=1 (add extra pod), maxUnavailable=1 (allow 1 pod down)

2 Q: How does Blue/Green Deployment work?

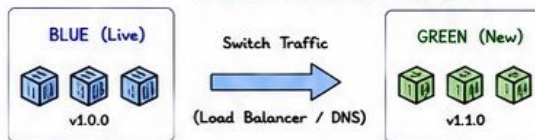
ANS:

- Two identical environments: Blue (current) and Green (new).
- Traffic is switched from Blue → Green after validation.
- Quick rollback by switching traffic back.

INTERVIEWER LOOKS FOR:

- Environment separation
- Traffic switching
- Rollback speed
- Risk reduction
- Production mindset

BLUE / GREEN DEPLOYMENT



Rollback = Switch traffic back to Blue

GOOD WHEN:

- ✓ Fast rollback needed
- ✓ High reliability
- ✓ Read-heavy apps
- ✓ Sufficient infra available

3 Q: What is Canary Release?

ANS:

- Release new version to a small subset of users first.
- Validate metrics and stability.
- Gradually increase traffic if healthy.
- If issues occur, stop rollout and rollback.

INTERVIEWER LOOKS FOR:

- Traffic splitting
- Risk management
- Metrics-driven decisions
- Rollback strategy
- User impact awareness

CANARY RELEASE FLOW



Monitor: Error rate, Latency, CPU, Business metrics

ROLLBACK IF:

- ✓ Error rate high
- ✓ Latency spike
- ✓ Business impact
- ✓ User complaints

4 Q: What is Progressive Delivery?

ANS:

- Automated process for releasing software in small steps.
- Uses automation, metrics, and quality gates.
- Includes canary, blue/green, feature flags.
- Goal: Safe, fast, and reliable releases.

INTERVIEWER LOOKS FOR:

- Automation mindset
- Quality gates
- Observability usage
- Continuous delivery
- Risk mitigation

PROGRESSIVE DELIVERY PIPELINE



If gate fails → Auto rollback or halt

5 Q: How do Feature Flags help in deployments?

ANS:

- Decouple code deployment from feature release.
- Hide or enable features without redeploying.
- Roll out to specific users or segments.
- Quick disable if issues occur.

INTERVIEWER LOOKS FOR:

- Decoupling releases
- Risk reduction
- User targeting
- Operational flexibility

FEATURE FLAG WORKFLOW



BEST FOR:

- ✓ New features
- ✓ Experiments
- ✓ A/B testing
- ✓ Gradual rollout

6 Q: How do you rollback a failed deployment?

ANS:

- Identify the failing release.
- Use kubectl rollout undo or Helm rollback.
- Verify previous version is stable.
- Communicate and run post-incident review.

INTERVIEWER LOOKS FOR:

- Rollback methods
- Impact assessment
- Communication
- Lessons learned
- Prevention thinking

ROLLBACK STRATEGIES



7 Q: How do you achieve Zero Downtime Deployments?

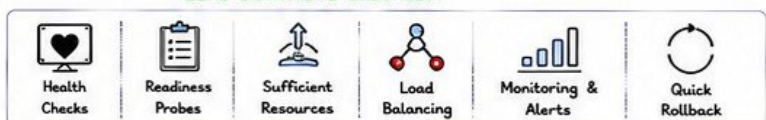
ANS:

- Use rolling updates, blue/green or canary.
- Health checks and readiness probes.
- Proper resource planning and auto scaling.
- Test thoroughly and monitor continuously.

INTERVIEWER LOOKS FOR:

- Strategy selection
- Health checks usage
- Capacity planning
- Monitoring focus
- Real-world execution

ZERO DOWNTIME CHECKLIST



KUBERNETES

```

kubectl get deploy
kubectl rollout status deploy/<name>
kubectl rollout history deploy/<name>
kubectl rollout undo deploy/<name>
kubectl describe pod <pod>
  
```

HELM

```

helm list
helm status <release>
helm history <release>
helm rollback <release> <rev>
helm upgrade <release> <chart>
  
```

METRICS TO WATCH

- Error Rate (5xx)
- Latency (p95/p99)
- CPU / Memory
- Request Rate
- Success Rate

TOOLS

- Prometheus
- Grafana
- Argo Rollouts
- Istio / Linkerd
- Datadog / New Relic

EVENTS TO CHECK

```

kubectl get events --sort-by=.lastTimestamp
kubectl describe pod <pod>
kubectl get pods -o wide
kubectl top pod, top node
  
```



PRO TIP

Great deployments are boring.
No drama for users, no surprises for engineers!



Golden Rule: Automate your deployments, observe everything, and always have a rollback plan.



INTERVIEWER WANTS TO SEE:

- ✓ Practical experience
- ✓ Trade-off thinking
- ✓ Communication skills
- ✓ Incident awareness
- ✓ Continuous improvement mindset

HELM PRODUCTION

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Helm Architecture
- ✓ Chart Design
- ✓ Values Management
- ✓ Template Debugging
- ✓ Dependency Management
- ✓ Helm Upgrades
- ✓ Rollback Scenarios

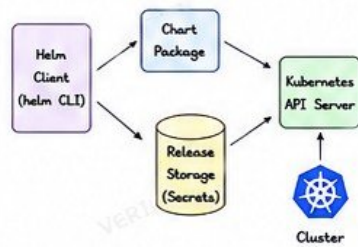
HELM IN PRODUCTION

- ★ Repeatable deployments
- ★ Versioned releases
- ★ Easy rollbacks
- ★ Kubernetes-native
- ★ Infrastructure as Code

1 HELM ARCHITECTURE

- Helm is a package manager for Kubernetes.
- It uses Charts to define, install and upgrade complex Kubernetes applications.
- Helm communicates with the Kubernetes API Server.
- Release stores are recorded in Secrets or ConfigMaps.

HELM ARCHITECTURE



2 CHART DESIGN

- A chart is a collection of files that describe a related set of Kubernetes resources.
- Every chart must have a Chart.yaml.
- Follow best practices for structure, reusability and maintainability.
- Use helpers, partials and library charts for clean templates.

CHART STRUCTURE

```

my-app/
├── Chart.yaml
├── values.yaml
├── charts/ (dependencies)
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── _helpers.tpl
│   └── ingress.yaml
├── README.md
└── values.schema.json (optional)
  
```

3 VALUES MANAGEMENT

- values.yaml holds default configuration.
- Users can override values at install/upgrade.
- Support multiple environments with separate values files.
- Use --set, --set-file and --set-string carefully.

VALUES HIERARCHY (LOW → HIGH PRIORITY)

Chart values.yaml	(defaults)
-f values-dev.yaml	(file override)
-f values-prod.yaml	(file override)
--set key=value	(CLI override)

```

helm install myapp ./mychart \
  -f values-common.yaml \
  -f values-prod.yaml \
  --set image.tag=1.2.3 \
  --set-string replicaCount="3"
  
```



BEST PRACTICE

Keep secrets out of values.yaml.
Use External Secrets, Sealed Secrets or SOPS.

4 TEMPLATE DEBUGGING

- Template errors are common. Helm provides powerful tools to debug.
- Always render templates locally before deploying.
- Use lint, template, and debug flags.

USEFUL COMMANDS

```

helm lint ./mychart
helm template myapp ./mychart
helm template myapp ./mychart -f values.yaml
helm template . --debug
helm get manifest myapp
helm get values myapp -a
  
```

COMMON TEMPLATE ISSUES



Syntax errors
(missing end)



Nil pointer
(e.g. .Values.foo)



Indentation
issues



Wrong data
type



Scope problems
(./ or \$)

```

helm default and required
replicas: {{ .Values.replicaCount | default 2 }}
image: {{ required "image.repository is required" .Values.image.repository }}
  
```

```

Use with and range carefully
{{- with .Values.resources }}
resources:
  {{ toYaml . | indent 2 }}
{{- end }}
  
```

5 DEPENDENCY MANAGEMENT

- Manage dependencies in Chart.yaml.
- Use helm dependency update to pull dependant charts.
- Use version constraints for stability.
- Lock dependencies with Chart.lock.

Chart.yaml (snippet)

```

dependencies:
- name: nginx-ingress
  version: 4.9.0
  repository: https://kubernetes.github.io/ingress-nginx
- name: prometheus
  version: 15.7.1
  repository: https://prometheus-community.github.io/helm-charts
  
```

```

helm dependency update ./mychart
helm dependency list ./mychart
  
```



TIP

Always commit Chart.lock for reproducible builds.

6 HELM UPGRADES

- Upgrade releases safely.
- Use --atomic to rollback on failure.
- Use --reuse-values to keep existing values.
- Use --history-max to limit history size.

UPGRADE COMMANDS

```

helm upgrade myapp ./mychart
helm upgrade myapp ./mychart -f values.yaml
helm upgrade myapp ./mychart --atomic
helm upgrade myapp ./mychart --reuse-values
helm upgrade myapp ./mychart --history-max 10
  
```

--atomic

If the upgrade fails, Helm automatically rolls back to the previous successful release.



7 ROLLBACK SCENARIOS

- Rollback when new release breaks.
- Rollback to specific revision.
- Rollback is instant and safe.
- Always verify after rollback.

ROLLBACK COMMANDS

```

helm history myapp
helm rollback myapp 2
helm rollback myapp 2 --wait
helm rollback myapp 2 --cleanup-on-fail
  
```

Check history to identify stable revision number.



COMMON ROLLBACK TRIGGERS

- Application error (5xx, crashes, timeouts)
- Resource limits exceeded (CPU/Memory)
- Bad configuration
- Dependency failure
- Security vulnerability

AFTER ROLLBACK

- ✓ Verify application is healthy
- ✓ Check logs and metrics
- ✓ Communicate to team
- ✓ Perform RCA
- ✓ Fix and test in lower env first

HELM CHEAT SHEET



Install

```

helm install myapp ./chart
helm install myapp ./chart -f values.yaml
  
```



Upgrade

```

helm upgrade myapp ./chart
helm upgrade myapp ./chart --atomic
  
```



Uninstall

```

helm uninstall myapp
helm uninstall myapp --keep-history
  
```



Status

```

helm status myapp
helm get all myapp
  
```



History

```

helm history myapp
  
```



Template

```

helm template myapp ./chart
helm template . --debug
  
```



Lint

```

helm lint ./chart
  
```



PRO TIP

Test charts in a sandbox cluster first.
Production incidents are expensive.



GOLDEN RULE

Always: Template → Lint → Test → Deploy → Verify → Monitor
Automate everything and always have a rollback plan.

GITOPS & ARGO CD

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ GitOps Workflow
- ✓ Desired State
- ✓ Drift Detection
- ✓ Argo CD Sync
- ✓ ApplicationSets
- ✓ Multi-cluster GitOps
- ✓ GitOps Troubleshooting

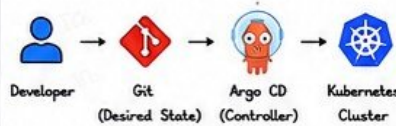
GITOPS PRINCIPLE

- Declarative.
- Versioned.
- Immutable.
- Automatically reconciled.
- Always converges to the desired state.

1 WHAT IS GITOPS?

- ANS:**
- GitOps is a way to deploy and manage infrastructure using Git as the single source of truth.
 - The desired state is stored in Git.
 - A controller continuously ensures the cluster matches Git.
 - Changes are made via pull requests and reviewed before merge.

GITOPS WORKFLOW



Argo CD continuously compares Git state with the live cluster and syncs automatically.

2 DESIRED STATE & DRIFT DETECTION

DESIRED STATE



What is defined in Git.

LIVE STATE



What is running in the cluster.

≠

DRIFT DETECTION

- Argo CD detects differences between Git and cluster.
- Causes of drift:
 - Manual changes
 - Out-of-band updates
 - Failed syncs
 - External controllers modifying resources

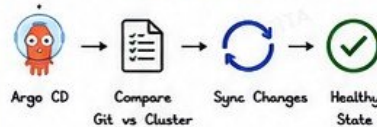


Goal: Always converge the live state to match Git.

3 ARGO CD SYNC

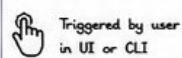
- ANS:**
- Sync brings the live state in the cluster to match the desired state in Git.
 - Argo CD supports automated and manual syncs.
 - Sync options:
 - Prune (remove deleted resources)
 - Self Heal (revert manual changes)
 - Sync Waves (ordered sync)
 - Sync Hooks (PreSync, PostSync)

SYNC FLOW



SYNC MODES

MANUAL SYNC



Triggered by user in UI or CLI

AUTOMATED SYNC



Automatically syncs on Git changes

4 APPLICATIONSETS

- ANS:**
- ApplicationSets generate multiple Argo CD Applications from templates.
 - Useful for managing deployments at scale.
 - Supports generators and templates.

COMMON GENERATORS



List Generator
Static list of clusters



Cluster Generator
All clusters from a cluster secret



Git Generator
Read directories from a repo



Matrix Generator
Combine multiple generators

APPLICATIONSET EXAMPLE

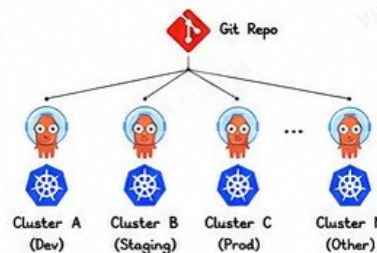
```

apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: my-apps
spec:
  generators:
    - list:
        elements:
          - cluster: dev
          - cluster: staging
          - cluster: prod
  template:
    metadata:
      name: my-app-{{cluster}}
    spec:
      project: default
      source:
        repoURL: https://github.com/veriqta/apps.git
        path: charts/my-app
        targetRevision: main
      destination:
        server: https://{{cluster-server}}
        namespace: my-app
  
```

5 MULTI-CLUSTER GITOPS

- ANS:**
- Single Git repo manages multiple clusters.
 - Argo CD can be per-cluster or a central Argo CD can manage many clusters.
 - Use cluster secrets to authenticate.
 - Ensure network connectivity and RBAC for each cluster.

MULTI-CLUSTER ARCHITECTURE



USEFUL ARGO CD COMMANDS



```

argocd app get <app>
argocd app sync <app>
argocd app diff <app>
argocd app history <app>
  
```

```

argocd app rollback <app> --revision <rev>
argocd app logs <app>
argocd app wait <app> --health --timeout 300
  
```

6 GITOPS TROUBLESHOOTING

- ANS:**
- Identify the issue.
 - Check Argo CD status.
 - Check diff and events.
 - Fix root cause.
 - Validate and sync.



Identify Issue



Check Status



Inspect Diff & Events



Fix Root Cause



Sync & Validate

7 COMMON GITOPS INTERVIEW QUESTIONS

- What is GitOps and how does it work?
- How does Argo CD detect drift?
- What are the benefits of GitOps?
- How would you rollback a bad deployment?
- How do you manage secrets in GitOps?
- What is an ApplicationSet?
- How do you handle multi-cluster GitOps?
- How do you debug a failed sync?

- What is the difference between sync and deploy?
- How do you prevent manual changes in the cluster?
- How do you handle breaking changes?

BEST PRACTICES

- ✓ Store everything in Git.
- ✓ Use automated sync with Self Heal.
- ✓ Protect main branch.
- ✓ Use PRs and reviews.
- ✓ Monitor Argo CD health.
- ✓ Keep secrets secure.



KEY TAKEAWAY



Git is the source of truth.
Argo CD is the enforcer.
Always converge to the desired state.



GITOPS GOLDEN RULES

- ✓ Declarative over imperative
- ✓ Automate everything
- ✓ Version everything
- ✓ Observe everything
- ✓ Secure everything



DON'T DO THIS

- ✗ Manual kubectl changes
- ✗ Editing in cluster
- ✗ Skipping reviews
- ✗ Storing secrets in Git
- ✗ Ignoring drift



INTERVIEWER WANTS TO SEE:

- ★ Real-world experience
- ★ Understanding of GitOps principles
- ★ Troubleshooting mindset
- ★ Automation and reliability thinking
- ★ Production-ready practices



PRO TIP

If you can explain how Git + Argo CD work together, you can explain 80% of GitOps in any interview.



GOLDEN REMINDER

Git is the memory.
Argo CD is the muscle.
Kubernetes is the execution.

AUTOMATE. ✓
OBSERVE. ✓
CONVERGE. ✓

STORAGE & STATEFUL APPLICATIONS

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Persistent Volumes
- ✓ Persistent Volume Claims
- ✓ Storage Classes
- ✓ StatefulSets
- ✓ CSI Drivers
- ✓ Backup Strategies
- ✓ Stateful Workload Recovery

STATEFUL APPS NEED:

- ★ Stable Identity
- ★ Stable Storage
- ★ Ordered Deployment
- ★ Reliable Recovery
- ★ Consistent Backups

1 PERSISTENT VOLUMES (PV)

- ANS:**
- PV is a piece of storage in the cluster provisioned by an admin or dynamically by StorageClass.
 - It is a cluster resource, independent of any Pod lifecycle.
 - Reclaim policies: Retain | Delete | Recycle (deprecated).



PV OBJECT (Example)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-01
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-0abc123
```

2 PERSISTENT VOLUME CLAIMS (PVC)

- ANS:**
- PVC is a request for storage by a user.
 - It consumes a PV that matches the request.
 - Pods mount PVCs, not PVs directly.
 - PVC is namespaced.



PVC OBJECT (Example)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: fast-ssd
```

3 STORAGE CLASSES

- ANS:**
- StorageClass defines how storage is provisioned.
 - Enables dynamic provisioning.
 - Parameters control type, IOPS, encryption, replication, etc.
 - Set a default StorageClass wisely.

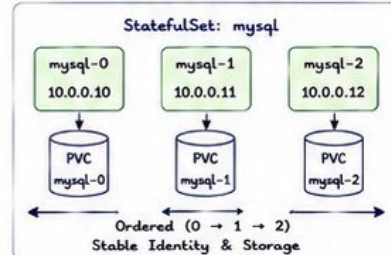
STORAGECLASS EXAMPLE

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  encrypted: "true"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

4 STATEFULSETS

- ANS:** Manages stateful applications.
- Each Pod gets a stable identity: <name>-<ordinal>
 - Stable network identity.
 - Stable storage (PVC per Pod).
 - Ordered & graceful deployment and scaling.

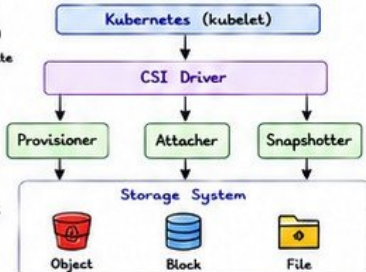
STATEFULSET DIAGRAM



5 CSI DRIVERS

- ANS:**
- Container Storage Interface (CSI) allows storage vendors to integrate with Kubernetes.
 - Types:
 - Block (e.g., EBS, GCE PD)
 - File (e.g., EFS, Filestore)
 - Object (e.g., S3 via gateway)
 - CSI = portability + extensibility.

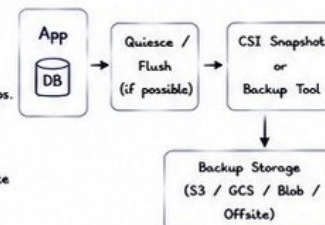
CSI ARCHITECTURE



6 BACKUP STRATEGIES

- ANS:** Application-aware backups for consistency.
- Use CSI Volume Snapshots for crash-consistent backups.
 - Schedule regular backups and test restores.
 - Store backups in a separate region or account.

BACKUP APPROACH

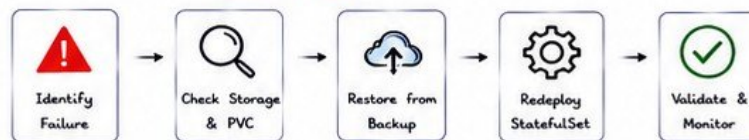


- TOOLS EXAMPLES**
- Velero
 - Kasten K10
 - AWS EBS Snapshot
 - Portworx
 - Restic + S3

7 STATEFUL WORKLOAD RECOVERY

- ANS:**
- Identify the failure (Pod, Node, Disk, or Region).
 - Verify PVC/PV health and attachment.
 - Restore from latest good backup or snapshot.
 - Scale up StatefulSet carefully.
 - Validate data integrity.
 - Monitor after recovery.

RECOVERY FLOW



GOLDEN TIP

Always assume failure: Disk, Node, AZ, Region, or Human.
Design for recovery from Day 1.

- Senior Engineers

COMMON INTERVIEW QUESTIONS

- Q1: What is the difference between PV and PVC?
Q2: How does dynamic provisioning work?
Q3: What is the purpose of StorageClass?
Q4: Why not use emptyDir for stateful apps?

- Q5: How does StatefulSet differ from Deployment?
Q6: What happens to PVC when Pod is deleted?
Q7: How do you backup a stateful workload?
Q8: How do you recover a corrupted database Pod?

- Q9: What is a CSI driver?
Q10: How do you migrate stateful apps safely?



PV RECLAIM POLICIES

- Retain → Keep PV after PVC deleted
- Delete → Delete underlying storage
- Recycle → (Deprecated) Basic scrub & reuse



ACCESS MODES

- RWO → ReadWriteOnce
- ROX → ReadOnlyMany
- RWX → ReadWriteMany



VOLUME BINDING MODES

- Immediate → Bind PV right away
- WaitForFirstConsumer → Wait until Pod is scheduled

BEST PRACTICES

- ✓ Use dedicated StorageClasses
- ✓ Set resource requests/limits
- ✓ Backup regularly and test
- ✓ Monitor storage metrics
- ✓ Document recovery runbooks



PRO TIP

Storage is not just disks. It's durability, performance, and recovery. Get it wrong → data loss and downtime.



GOLDEN RULE

Stateful apps are only as reliable as their storage and backups.

- INTERVIEWER WANTS TO SEE:**
- ✓ Real experience with storage systems
 - ✓ Understanding of failure & recovery
 - ✓ Backup strategy and operational maturity



OBSERVABILITY IN KUBERNETES

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Metrics Collection
- ✓ Prometheus Operator
- ✓ kube-state-metrics
- ✓ Logging Architecture
- ✓ Distributed Tracing
- ✓ AlertManager
- ✓ Production Dashboards

OBSERVABILITY PRINCIPLES

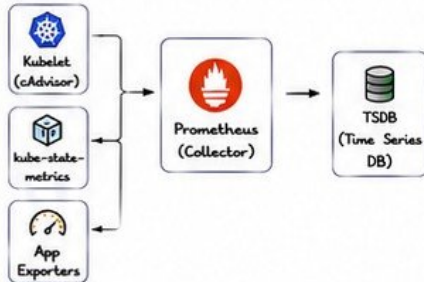
- ★ Understand what is happening.
- ★ Detect issues early.
- ★ Investigate fast.
- ★ Improve continuously.

1 METRICS COLLECTION

METRICS FLOW

ANS: Metrics show the health and performance of your cluster and apps.

- Collected from nodes, pods, services, and applications.
- Pulled by Prometheus at regular intervals.
- Use labels for rich querying.



COMMON METRICS

- CPU / Memory usage
- Pod restarts
- Request / Error rate
- Latency (p95, p99)
- Saturation
- Queue length
- Custom business metrics

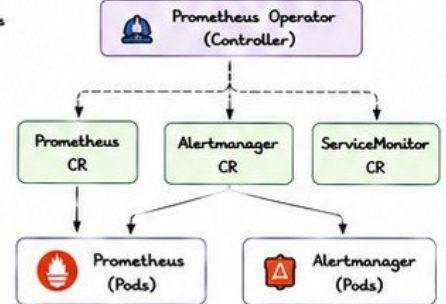


2 PROMETHEUS OPERATOR

OPERATOR ARCHITECTURE

ANS: Operator manages Prometheus instances as Kubernetes custom resources.

- Simplifies deployment, configuration, scaling, and upgrades.
- Uses CRDs: Prometheus, ServiceMonitor, PodMonitor, Alertmanager.
- Ensures HA and best practices by default.



The Operator turns your desired state into running monitoring components.

3 KUBE-STATE-METRICS

EXPOSED METRICS EXAMPLES

ANS: Exposes Kubernetes object metrics via Prometheus.

- Example: deployments, pods, jobs, nodes, secrets, configmaps.
- Critical for understanding cluster state, not just resource usage.

- kube_deployment_status_replicas_available
- kube_pod_status_phase
- kube_node_status_condition
- kube_job_status_succeeded
- kube_persistentvolumeclaim_status_phase



4 LOGGING ARCHITECTURE

LOGGING PIPELINE EXAMPLE

ANS: Logs are collected, aggregated, stored, and visualized.

- Use a cluster-wide logging solution.
- Fluentd / Fluent Bit ships logs.
- Central storage: Elasticsearch / OpenSearch.
- Visualization: Kibana / Grafana / Loki.



BEST PRACTICES

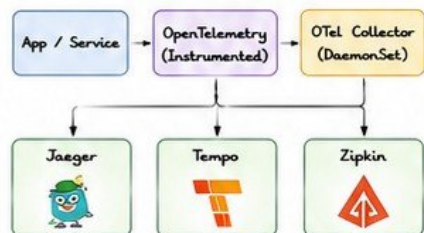
- ✓ Include cluster, namespace, app labels
- ✓ Use structured logs (JSON)
- ✓ Set log retention policies
- ✓ Mask sensitive data

5 DISTRIBUTED TRACING

TRACING ARCHITECTURE

ANS: Tracing shows the flow of requests across services.

- Use OpenTelemetry instrumentation.
- Collector receives spans and exports to backend.
- Backends: Jaeger, Tempo, Zipkin.

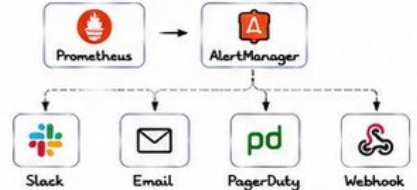


6 ALERTMANAGER

ALERTMANAGER FLOW

ANS: Handles alerts from Prometheus.

- Deduplicates, groups, and routes alerts.
- Supports notifications: Email, Slack, PagerDuty, Webhook, OpsGenie.
- Silences and inhibition reduce noise.



GOOD ALERTS

- ✓ Actionable
- ✓ Clear
- ✓ Low noise
- ✓ With runbook link

AVOID

- ✗ Too many alerts
- ✗ Vague messages
- ✗ No owner
- ✗ No runbook

7 PRODUCTION DASHBOARDS

DASHBOARD CATEGORIES

ANS: Dashboards turn metrics into actionable insights.

- Build role-based dashboards.
- Keep them simple, fast, and useful.



DASHBOARD BEST PRACTICES

- ✓ Start with SLIs and SLOs
- ✓ Show rates, not just totals
- ✓ Use clear panels and thresholds
- ✓ Provide drill-down links
- ✓ Review dashboards regularly



COMMON OBSERVABILITY INTERVIEW QUESTIONS

- Q1: How does Prometheus collect metrics?
 Q2: What is the role of the Prometheus Operator?
 Q3: Why is kube-state-metrics important?
 Q4: How would you design a logging solution for a Kubernetes cluster?

- Q5: How does distributed tracing help in debugging?
 Q6: What is the difference between logs, metrics, and traces?
 Q7: How does AlertManager reduce alert fatigue?

- Q8: How do you create an alert for high latency?
 Q9: How do you troubleshoot missing metrics?
 Q10: What dashboards would you build for production?



PRO TIP

You can't improve what you can't observe. Instrument, collect, alert, learn, and iterate.



GOLDEN RULE

Instrument everything important. Alert on user impact, not resource usage.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Troubleshooting mindset
- ✓ Understanding of trade-offs
- ✓ Production-ready thinking

OBSERVABILITY = VISIBILITY

Visibility leads to confidence. Confidence leads to stability. Stability leads to happy users.



KUBERNETES SECURITY

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ RBAC
- ✓ Service Accounts
- ✓ Network Policies
- ✓ Pod Security Standards
- ✓ Admission Controllers
- ✓ Image Security
- ✓ Secrets Management

SECURITY IS A LAYER,
NOT A PRODUCT.

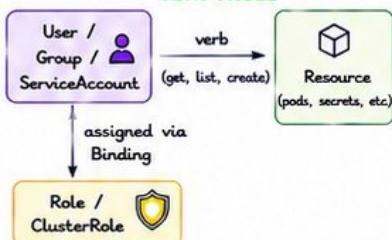
- ★ Least privilege
- ★ Assume breach
- ★ Validate everything
- ★ Monitor always
- ★ Automate security

1 RBAC (ROLE-BASED ACCESS CONTROL)

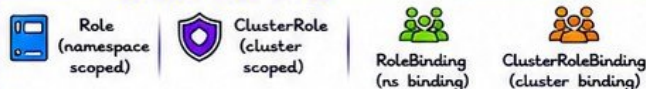
ANS: Controls who can do what in the cluster.

- Uses Roles/ClusterRoles to define permissions.
- Binding users or service accounts to roles.
- Follow least privilege principle.

RBAC MODEL



COMMON ROLE TYPES



2 SERVICE ACCOUNTS

SERVICE ACCOUNT FLOW

ANS: Identities for processes running in Pods.

- Automatically mounted in Pods.
- Should have minimal permissions.
- Avoid using default service account.



BEST PRACTICES

- ✓ Create dedicated ServiceAccount per application.
- ✓ Do not grant cluster-admin to ServiceAccounts.
- ✓ Disable automountServiceAccountToken if not needed.
- ✓ Rotate tokens and use short-lived tokens (Bound SA Tokens).

3 NETWORK POLICIES

ANS: Control traffic flow between Pods and namespaces.

- Default is allow all (unless policy exists).
- Applied using CNI plugin (Calico, Cilium, etc.).
- Use to implement zero trust networking.

NETWORK POLICY EXAMPLE

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
  
```



4 POD SECURITY STANDARDS (PSS)

PSS ENFORCEMENT

ANS: Defines security levels for Pods.

- **Privileged** (least secure)
- **Baseline** (restricted capabilities)
- **Restricted** (most secure)
- Enforced via labels on namespaces.
- Replaces PSP (deprecated).

LEVEL	KEY RESTRICTIONS
Privileged	Allow privilege escalation, host namespaces, hostPath, any capabilities.
Baseline	Drop most capabilities, disallow privilege escalation, limited host access.
Restricted	Run as non-root, read-only root FS, drop all capabilities, no host namespaces.

BEST PRACTICES

- ✓ Use Restricted for all production namespaces.
- ✓ Set enforce: restricted and audit: restricted.
- ✓ Continuously monitor and remediate violations.

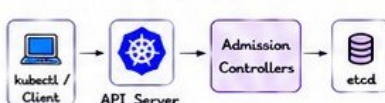


5 ADMISSION CONTROLLERS

ANS: Intercept requests to the API server.

- Allow, modify, or reject resources.
- Used to enforce policies and security rules.
- Examples: OPA/Gatekeeper, Kyverno, Pod Security, ImagePolicyWebhook.

ADMISSION FLOW



USE CASES

- ✓ Enforce labeling standards
- ✓ Block privileged containers
- ✓ Validate images and registries
- ✓ Enforce resource limits

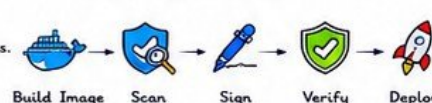


6 IMAGE SECURITY

IMAGE SECURITY PIPELINE

ANS: Use trusted base images.

- Scan images for vulnerabilities.
- Sign images and verify signatures.
- Use image policies to block untrusted images.
- Keep images updated.



BEST PRACTICES

- ✓ Use minimal and official base images.
- ✓ Scan in CI/CD and block high/critical CVEs.
- ✓ Use cosign/Notary for image signing.
- ✓ Enforce ImagePolicyWebhook or Kyverno policies.

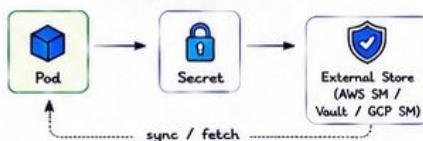


7 SECRETS MANAGEMENT

ANS: Store sensitive data in Secrets.

- Never store secrets in images or code.
- Use external secret stores for production.
- Encrypt at rest and control access via RBAC.

SECRETS ARCHITECTURE



BEST PRACTICES

- ✓ Use short-lived secrets.
- ✓ Rotate regularly.
- ✓ Limit secret access.
- ✓ Use ExternalSecrets or CSI drivers.



COMMON KUBERNETES SECURITY INTERVIEW QUESTIONS

- Q1: What is RBAC and how does it work?
Q2: How do you restrict access to a namespace?
Q3: What is the purpose of ServiceAccounts?
Q4: How do Network Policies work?

- Q5: What are Pod Security Standards?
Q6: How do Admission Controllers enhance security?
Q7: How do you prevent running privileged containers?

- Q8: How do you scan and secure container images?
Q9: How do you manage and rotate secrets?
Q10: What is the least privilege principle in K8s?



PRO TIP

Security is not a checklist. It's a continuous practice. Automate. Monitor. Improve.



GOLDEN RULE

Least privilege. Always validate. Never trust. Always verify.



INTERVIEWER WANTS TO SEE:
Real-world experience.
Understanding of trade-offs.
Security mindset.
Clear communication.



SECURE BY DEFAULT

Build secure. Run secure. Stay secure. That's how clusters survive.

PRODUCTION SCALING

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Horizontal Pod Autoscaler
- ✓ Vertical Pod Autoscaler
- ✓ Cluster Autoscaler
- ✓ Resource Requests
- ✓ Resource Limits
- ✓ Scaling Bottlenecks
- ✓ Capacity Planning

SCALING IS A DESIGN DECISION, NOT A REACTION.

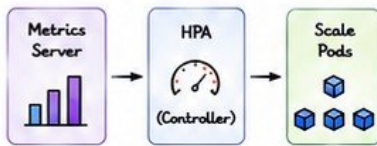
- ★ Measure before you scale.
- ★ Scale early, not late.
- ★ Automate what you can.
- ★ Understand your limits.
- ★ Plan for growth.

1 HORIZONTAL POD AUTOSCALER (HPA)

ANS: Scales the number of Pods based on metrics.

- Most common metric: CPU utilization.
- Can use custom metrics and external metrics.
- Works with Deployments, StatefulSets and others.

HPA WORKFLOW



EXAMPLE HPA (CPU BASED)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  
```

BEST PRACTICES

- ✓ Set proper min and max replicas.
- ✓ Use meaningful metrics.
- ✓ Test scaling before production.
- ✓ Avoid flapping with stabilization windows.

2 VERTICAL POD AUTOSCALER (VPA)

ANS: Adjusts CPU and memory requests/limits for Pods.

- Recommends or updates resource values.
- Does NOT change the number of Pods.
- Helps right-size workloads.

VPA MODES

Off	Analysis only. No changes made.
Initial	Sets resources at pod creation.
Auto	Updates resources and restarts Pods when needed.

VPA ARCHITECTURE



BEST PRACTICES

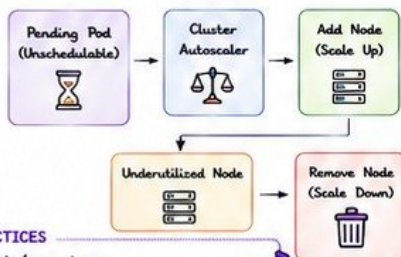
- ✓ Set VPA with caution in production.
- ✓ Test with OFF mode first.
- ✓ Combine with HPA carefully (HPA for scale, VPA for size).

3 CLUSTER AUTOSCALER (CA)

ANS: Automatically adjusts the number of nodes in the cluster.

- Adds nodes when Pods are pending.
- Removes nodes when they are underutilized.
- Works with Cloud provider APIs.

CA FLOW



BEST PRACTICES

- ✓ Set proper node group min/max sizes.
- ✓ Use multiple node groups for different workloads.
- ✓ Ensure pods have requests set (critical for scale decisions).
- ✓ Use PDBs to avoid disruptions during scale down.

4 RESOURCE REQUESTS vs LIMITS

ANS: Requests = What a container needs to run.

- Limits = Maximum it can consume.
- Requests affect scheduling.
- Limits prevent noisy neighbor problems.

EXAMPLE

```

resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
  
```

BEST PRACTICES

- ✓ Always set both requests and limits.
- ✓ Start small and adjust based on metrics.
- ✓ Avoid requests > limits.

5 SCALING BOTTLENECKS

ANS: CPU bottleneck

- Memory bottleneck
- I/O bottleneck
- Database bottleneck
- Network bottleneck
- Upstream dependency bottleneck
- Application design bottleneck

BOTTLENECK IDENTIFICATION



BEST PRACTICES

- ✓ Instrument your application.
- ✓ Monitor end-to-end.
- ✓ Understand saturation points.

6 CAPACITY PLANNING

ANS: Plan for growth before you need it.

- Use historical data and trends.
- Consider traffic growth, seasonality, and releases.
- Plan for failures too.

CAPACITY PLANNING PROCESS



KEY CONSIDERATIONS

- ✓ Traffic growth
- ✓ User behavior
- ✓ Application releases
- ✓ Infrastructure limits
- ✓ Cost optimization
- ✓ SLA requirements
- ✓ Disaster recovery
- ✓ Team and process

7 COMMON PRODUCTION INTERVIEW QUESTIONS

- Q1: How does HPA work internally?
 Q2: When would you use VPA?
 Q3: Can HPA and VPA be used together?
 Q4: How does Cluster Autoscaler decide to add a node?
 Q5: What happens if pods don't have resource requests?
 Q6: How do you troubleshoot HPA not scaling?

- Q7: What are common scaling bottlenecks?
 Q8: How do you plan capacity for Black Friday traffic?
 Q9: How do you prevent over-scaling?
 Q10: How do you test scaling before production?



PRO TIP

Scaling is not magic. It's engineering backed by data, tests, and experience. Measure. Plan. Automate. Improve.



GOLDEN RULE

Right-size your workload. Automate scaling. Monitor continuously. Optimize relentlessly.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Deep understanding
- ✓ Clear communication
- ✓ Trade-off analysis



SCALE SMART, NOT FAST

Scale with confidence. Scale with metrics. Scale for reliability. Scale for your users.

= SERVICE MESH =

INTERVIEW QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

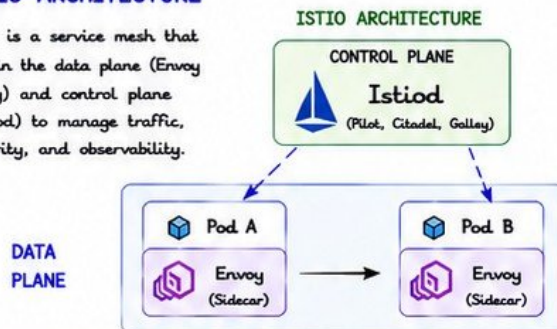
- ✓ Istio Architecture
- ✓ Envoy Proxy
- ✓ Traffic Management
- ✓ mTLS
- ✓ Service Discovery
- ✓ Observability
- ✓ Production Use Cases

SERVICE MESH
ISN'T MAGIC.
IT'S ENGINEERING.

- ★ Secure
- ★ Observable
- ★ Reliable
- ★ Scalable

1 ISTIO ARCHITECTURE

ANS: Istio is a service mesh that sits in the data plane (Envoy proxy) and control plane (Istiod) to manage traffic, security, and observability.

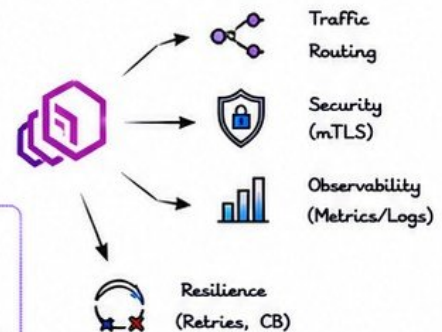


- Control Plane: Handles config, policies, certificates.
- Data Plane: Envoy sidecars handle all traffic.

2 ENVOY PROXY

ANS: Envoy is a high-performance Layer 7 proxy used as the data plane in Istio. Deployed as a sidecar next to every workload.

ENVOY RESPONSIBILITIES



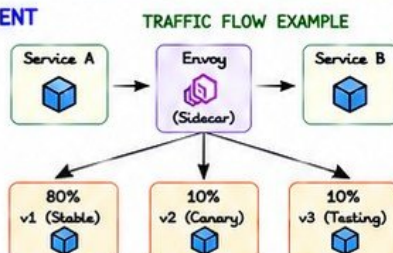
KEY BENEFITS

- ✓ Language agnostic
- ✓ High performance
- ✓ Rich observability
- ✓ Extensible via filters (Lua, WASM)

3 TRAFFIC MANAGEMENT

ANS: Control how traffic flows between services.

- Routing
- Load Balancing
- Retries
- Timeouts
- Fault Injection
- Traffic Splitting
- Canary & Blue/Green



BEST PRACTICES

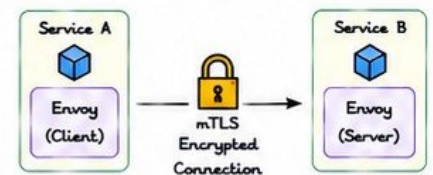
- ✓ Start with small percentage for canary.
- ✓ Use retries/timeouts for resilience.
- ✓ Always validate before 100% rollout.

4 mTLS (MUTUAL TLS)

ANS: Provides service-to-service authentication and encryption.

- Automatic certificate management
- Identity based access
- Zero trust communication
- Strong security by default

mTLS FLOW



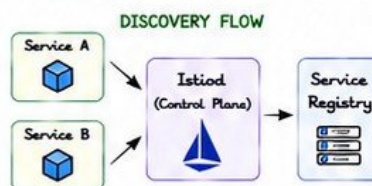
BEST PRACTICES

- ✓ Enable mTLS mesh-wide.
- ✓ Use STRICT mode in production.
- ✓ Rotate certificates automatically.

5 SERVICE DISCOVERY

ANS: Istio uses the control plane (Istiod) for service discovery and configuration.

- Automatic registration of services
- Health checking
- Load balancing integration



BEST PRACTICES

- ✓ Let Istio manage service discovery.
- ✓ Use Kubernetes DNS for names.
- ✓ Avoid hardcoded endpoints.

6 OBSERVABILITY

ANS: Istio provides deep observability out of the box.

- Metrics (Prometheus)
- Logs (ELK/Loki)
- Traces (Jaeger)
- Kiali dashboards
- Grafana dashboards

OBSERVABILITY STACK



BEST PRACTICES

- ✓ Monitor RED (Rate, Errors, Duration).
- ✓ Create SLOs and alerts.
- ✓ Use Kiali for mesh insights and debugging.

7 PRODUCTION USE CASES

ANS: Service mesh is used in real production for:

- Microservices communication
- Security & compliance
- Traffic control at scale
- Multi-cluster & multi-cloud
- Observability at scale
- Resiliency and fault tolerance

COMMON USE CASES



8 COMMON INTERVIEW QUESTIONS

- Q1: What is a service mesh?
Q2: How does Istio architecture work?
Q3: What is Envoy proxy?
Q4: How does mTLS work in Istio?
- Q5: How is traffic routing managed?
Q6: How does service discovery work?
Q7: What observability tools does Istio provide?



PRO
TIP

Start simple.
Observe deeply.
Automate everything.
Trust but verify.



GOLDEN RULE
Secure by default.
Observe continuously.
Automate and
test continuously.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Understanding of trade-offs
- ✓ Security mindset
- ✓ Production readiness



REMEMBER
A service mesh is not a product.
It's a platform. Design it,
simplify it, Design it,
observe it, then scale it.

MULTI-CLUSTER KUBERNETES

INTERVIEW TOPICS

Senior interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Multi-cluster Design
- ✓ Federation Concepts
- ✓ Cross-cluster Networking
- ✓ Disaster Recovery
- ✓ Global Load Balancing
- ✓ Fleet Management
- ✓ Enterprise Architectures

ENTERPRISE THINKING

Multi-cluster is about resilience, scale, isolation and control.

Design it right.
Operate it smart.

1 MULTI-CLUSTER DESIGN

ANS: Multi-cluster provides isolation, resilience, compliance and scale beyond a single cluster.

COMMON DESIGN PATTERNS

 Multi-Cluster per Region	One cluster per region for low latency and failure isolation.
 Multi-Cluster per Team	Teams own their clusters. Improves security and autonomy.
 Multi-Cluster per Environment	Separate clusters for dev, staging, prod for strong isolation.

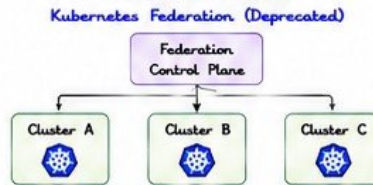
BEST PRACTICES

- ✓ Isolate failure domains.
- ✓ Standardize cluster configurations.
- ✓ Automate provisioning and lifecycle.
- ✓ Centralize observability and policy.

2 FEDERATION CONCEPTS

ANS: Federation connects multiple clusters under one control plane for resource management and governance.

FEDERATION MODELS



Modern Alternatives

- Karmada (CNCF)
- Rancher Fleet
- Open Cluster Management (ACM)
- GitOps (Argo CD + ApplicationSet)

BEST PRACTICES

- ✓ Use GitOps for consistent deployments.
- ✓ Define cluster placement policies.
- ✓ Enforce governance and security.
- ✓ Monitor from a central platform.

3 CROSS-CLUSTER NETWORKING

ANS: Enables services in different clusters to communicate securely.

APPROACHES

 Service Mesh	Istio, Linkerd, or Consul mesh for cross-cluster service discovery and traffic management.
 Cluster Mesh	Cilium Cluster Mesh or KubeSphere for network-level connectivity.
 VPN / VPC Peering	Connect clusters via private networks (AWS TGW, VPN, Azure VNet).

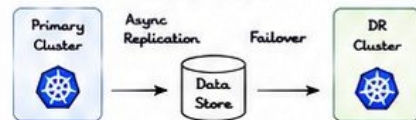
BEST PRACTICES

- ✓ Use service mesh for L7 communication.
- ✓ Encrypt traffic across clusters.
- ✓ Limit cross-cluster dependencies.
- ✓ Monitor latency and reliability.

4 DISASTER RECOVERY

ANS: Multi-cluster improves resilience and enables faster recovery.

DR STRATEGIES



- ✓ Replicate data across regions.
- ✓ Use RPO and RTO targets.
- ✓ Automate failover and fallback.
- ✓ Test DR regularly.

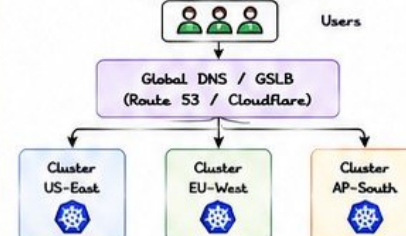
KEY POINT

DR is not just backups. It's people, process and regular testing.

5 GLOBAL LOAD BALANCING

ANS: Distributes traffic across clusters and regions for high availability and performance.

ARCHITECTURE EXAMPLE



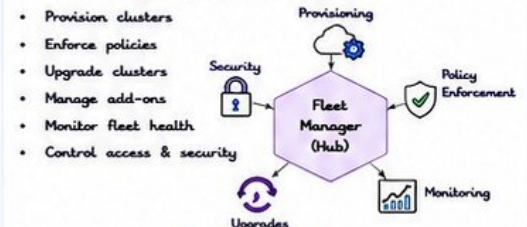
BEST PRACTICES

- ✓ Use health checks and latency routing.
- ✓ Design for regional failover.
- ✓ Keep data close to users.
- ✓ Monitor global traffic patterns.

6 FLEET MANAGEMENT

ANS: Manages multiple clusters at scale with consistency and automation.

FLEET MANAGEMENT TASKS



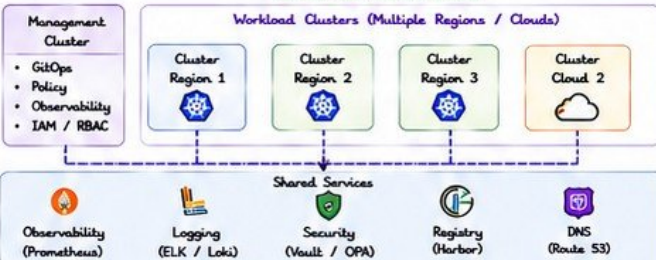
COMMON TOOLS

 Rancher Fleet	 Open Cluster Management	 Karmada	 Anthos	 Argo CD AppSet
---	---	---	--	--

7 ENTERPRISE ARCHITECTURES

ANS: A typical enterprise multi-cluster architecture.

REFERENCE ARCHITECTURE



ENTERPRISE PRINCIPLES

- ✓ Standardize everything.
- ✓ Automate everything.
- ✓ Secure by default.
- ✓ Observe everything.
- ✓ Test for failure.

8 COMMON INTERVIEW QUESTIONS

- Q1: Why would you choose multi-cluster over a single large cluster?
- Q2: How do you connect services across clusters?
- Q3: What is the role of a service mesh in multi-cluster environments?
- Q4: How do you handle disaster recovery in multi-cluster setups?
- Q5: How do you manage policies across multiple clusters?
- Q6: What are the challenges of multi-cluster operations?
- Q7: How do you monitor multiple clusters centrally?
- Q8: How do you ensure security in a multi-cluster architecture?



PRO TIP

Start small. Automate early. Standardize always. Test continuously.



GOLDEN RULE

Design for failure. Automate recovery. Observe everything. Keep users happy.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Architecture thinking
- ✓ Problem solving
- ✓ Production mindset



REMEMBER

Multi-cluster is complex but worth it. Resilience is built, not bought.

= CLOUD NATIVE ARCHITECTURE =

INTERVIEW QUESTIONS

Senior interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Twelve-Factor App
- ✓ Microservices
- ✓ Event-driven Systems
- ✓ API Gateway
- ✓ Service Communication
- ✓ Resilience Patterns
- ✓ Cloud-native Design

CLOUD NATIVE MINDSET

Build systems that are:

- ★ Scalable
- ★ Resilient
- ★ Observable
- ★ Secure
- ★ Automated
- ★ Evolvable

1 TWELVE-FACTOR APP

ANS: A set of best practices for building SaaS apps that are scalable, resilient and easy to operate.

THE 12 FACTORS

- 1 Codebase
- 2 Dependencies
- 3 Config
- 4 Backing Services
- 5 Build, Release, Run
- 6 Processes
- 7 Port Binding
- 8 Concurrency
- 9 Disposability
- 10 Dev/Prod Parity
- 11 Logs
- 12 Admin Processes



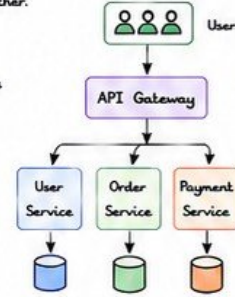
BEST PRACTICES

- ✓ Store config in environment
- ✓ One process per container
- ✓ Logs to stdout
- ✓ Stateless & disposable

2 MICROSERVICES

ANS: Breaking down applications into small, independent services that work together.

EXAMPLE ARCHITECTURE



KEY CHARACTERISTICS

- Small and focused services
- Independent deployment
- Decentralized data
- Technology agnostic
- Fault isolation
- Owned by small teams

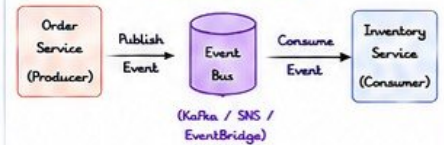
BEST PRACTICES

- ✓ Design for loose coupling
- ✓ Use contracts between services
- ✓ Automate everything
- ✓ Monitor and log each service

3 EVENT-DRIVEN SYSTEMS

ANS: Components communicate by publishing and consuming events asynchronously.

EVENT FLOW EXAMPLE



BENEFITS

- Decoupled services
- Scalable and resilient
- Real-time data flow
- Easy to extend

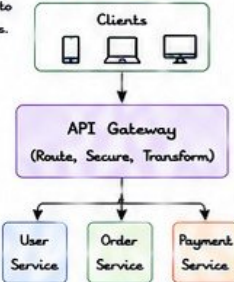
BEST PRACTICES

- ✓ Use durable event stores
- ✓ Design idempotent consumers
- ✓ Handle retries and failures
- ✓ Monitor event lag

4 API GATEWAY

ANS: A single entry point for clients to access multiple backend services. Handles routing, security and traffic management.

API GATEWAY ARCHITECTURE



RESPONSIBILITIES

- Request routing
- Authentication & authorization
- Rate limiting & throttling
- Request/response transformation
- Caching
- Monitoring & logging

BEST PRACTICES

- ✓ Secure all endpoints
- ✓ Use rate limiting
- ✓ Cache where appropriate
- ✓ Monitor for anomalies

5 SERVICE COMMUNICATION

ANS: How microservices communicate with each other.

SYNCHRONOUS

(Request/Response)

HTTP / REST

gRPC



Real-time response needed

ASYNCHRONOUS

(Event Messaging)

Kafka

RabbitMQ

SNS/SQS



Decoupled & scalable

BEST PRACTICES

- ✓ Use sync for short, real-time calls
- ✓ Use async for long-running tasks
- ✓ Implement timeouts and retries
- ✓ Use circuit breakers

6 RESILIENCE PATTERNS

ANS: Patterns to build systems that can handle failures gracefully.

	Retry	Retry failed requests with backoff.
	Circuit Breaker	Stop calling a failing service temporarily.
	Timeout	Fail fast if a service takes too long.
	Bulkhead	Isolate resources to prevent cascading failures.
	Fallback	Provide alternative response when failing.

BEST PRACTICES

- ✓ Combine patterns for strong resilience
- ✓ Test failure scenarios
- ✓ Monitor and alert on failures

7 CLOUD-NATIVE DESIGN PRINCIPLES

ANS: Design principles for building modern cloud-native applications.

Scalability



Scale out horizontally.

Resilience



Design for failure.

Observability



Logs, metrics and traces everywhere.

Automation



Automate build, deploy and operations.

Portability



Run anywhere (cloud, on-prem, edge).

Security



Security by default.

Evolutionary



Make small changes, iterate continuously.

BEST PRACTICES

- ✓ Embrace containers and Kubernetes
- ✓ Use Infrastructure as Code
- ✓ Implement CI/CD pipelines
- ✓ Keep services loosely coupled
- ✓ Continuously monitor and improve

8 COMMON INTERVIEW QUESTIONS

Q1: What is a cloud-native application?

Q2: Explain the Twelve-Factor App.

Q3: How do microservices differ from monoliths?

Q4: What are the benefits of event-driven architecture?

Q5: What is the role of an API Gateway?

Q6: When would you use gRPC instead of REST?

Q7: What resilience patterns have you implemented?

Q8: How do you design a cloud-native system for scale?



PRO TIP

Design for change. Build systems that can evolve with your business.



GOLDEN RULE

Automate everything. Observe everything. Secure everything. Document everything.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Architectural thinking
- ✓ Problem solving
- ✓ Production readiness



REMEMBER

Cloud-native is not a goal. It's a mindset. Ship small, measure, learn and improve continuously.

ENTERPRISE KUBERNETES SYSTEM DESIGN

INTERVIEW TOPICS

Senior interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Design a Production Platform
- ✓ Namespace Strategy
- ✓ Security Boundaries
- ✓ Multi-Tenant Clusters
- ✓ Cluster Isolation
- ✓ High Optimization

ENTERPRISE THINKING

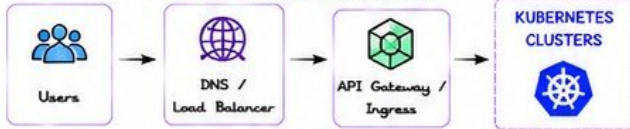
- Design for scale.
- Design for security.
- Design for reliability.
- Design for cost.

★ Design for the business.

1 DESIGN A PRODUCTION PLATFORM

ANS: A production platform must be secure, reliable, scalable, observable, and easy to operate.

PLATFORM ARCHITECTURE OVERVIEW



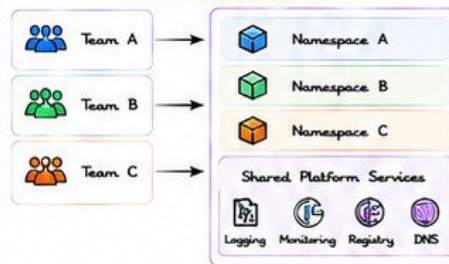
PLATFORM COMPONENTS

- Identity & Access (OIDC, RBAC, SSO)
- Cluster Management
- Networking (CNI, Ingress, DNS)
- Security (Pod Security, Network Policies)
- Observability (Logs, Metrics, Traces)
- CI/CD Integration
- Backup & Disaster Recovery
- Cost & Resource Management

2 MULTI-TENANT CLUSTERS

ANS: Multiple teams or business units share the same cluster while remaining isolated and secure.

TENANT ISOLATION MODEL



BEST PRACTICES

- ✓ Use Namespaces for isolation
- ✓ Enforce Resource Quotas
- ✓ Use Network Policies
- ✓ RBAC per Namespace
- ✓ Limit cross-namespace access

3 NAMESPACE STRATEGY

ANS: Namespaces provide logical isolation for teams, apps, and environments.

NAMESPACE STRUCTURE EXAMPLE



BEST PRACTICES

- ✓ One namespace per team or application.
- ✓ Separate environments with namespaces.
- ✓ Avoid too many namespaces (overhead).
- ✓ Use naming conventions (team-env-purpose).



4 CLUSTER ISOLATION

ANS: Use multiple clusters to isolate sensitive workloads, compliance requirements, or environments.

ISOLATION LEVELS



BEST PRACTICES

- ✓ Separate clusters for Prod and Non-Prod
- ✓ Isolate sensitive data workloads
- ✓ Different clusters for different compliance requirements
- ✓ Use cluster policies and guardrails

5 SECURITY BOUNDARIES

ANS: Define and enforce strong security boundaries across tenants and workloads.

SECURITY LAYERS



BEST PRACTICES

- ✓ Enforce least privilege access.
- ✓ Prevent pod-to-pod communication by default.
- ✓ Scan images and block vulnerable ones.
- ✓ Centralize logs and audit everything.



6 HIGH AVAILABILITY

ANS: Design for zero-single-point-of-failure across the entire platform.

HIGH AVAILABILITY AREAS



BEST PRACTICES

- ✓ Use managed control plane (EKS, GKE, AKS)
- ✓ Spread nodes across AZs
- ✓ Back up etcd regularly
- ✓ Use Pod Disruption Budgets (PDBs)
- ✓ Test failover regularly

7 COST OPTIMIZATION

ANS: Optimize resources, scale intelligently, and eliminate waste.

COST OPTIMIZATION STRATEGIES



BEST PRACTICES

- ✓ Request what you need, not the maximum.
- ✓ Use VPA/HPA for efficient scaling.
- ✓ Use Spot/Preemptible for non-critical workloads.
- ✓ Monitor and allocate costs by team.



8 ENTERPRISE KUBERNETES DESIGN PRINCIPLES



COMMON INTERVIEW QUESTIONS

- Q1: How would you design a multi-tenant Kubernetes platform?
- Q2: How do you isolate workloads in a shared cluster?
- Q3: What would you choose: multiple clusters or namespaces?
- Q4: How do you design for high availability in Kubernetes?
- Q5: How do you reduce Kubernetes costs?
- Q6: How do you secure a Kubernetes platform?
- Q7: How do you handle disaster recovery?
- Q8: How do you manage upgrades safely?



PRO TIP

Good design isn't about tools, it's about trade-offs. Understand the WHY.



GOLDEN RULE
Reliability first. Security always. Cost matters. Automate everything.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Architectural thinking
- ✓ Trade-off analysis
- ✓ Risk awareness



REMEMBER
Kubernetes is a platform. Your job is to design systems that are secure, reliable, and scalable.

PRODUCTION INCIDENT SCENARIOS

INTERVIEW TOPICS

Senior interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Cluster Outage
- ✓ Node Failures
- ✓ API Server Unavailable
- ✓ etcd Corruption
- ✓ DNS Failures
- ✓ Ingress Outages
- ✓ Recovery Strategy

ENTERPRISE MINDSET

- ★ Detect Fast
- ★ Communicate Clearly
- ★ Act Deliberately
- ★ Verify Everything
- ★ Learn & Improve

1 CLUSTER OUTAGE



SYMPTOMS

- All pods unavailable
- kubectl commands failing
- Applications down

ROOT CAUSES

- Control plane failure
- Network partition
- Infra / cloud outage

INVESTIGATION COMMANDS

```
> kubectl cluster-info
> kubectl get nodes
> kubectl get pods --all-namespaces
> ping <node-ip>
```

RECOVERY STRATEGY

- ✓ Identify failure domain
- ✓ Restore control plane
- ✓ Verify etcd health
- ✓ Bring up worker nodes
- ✓ Validate workloads

2 NODE FAILURES



SYMPTOMS

- Pods on node become NotReady
- Applications degrade
- New pods not scheduled

ROOT CAUSES

- Hardware failure
- Disk pressure / full
- Kubelet / runtime crash
- Network issue

INVESTIGATION COMMANDS

```
> kubectl get nodes
> kubectl describe node <node>
> journalctl -u kubelet -f
> df -h /
> top / htop
```

RECOVERY STRATEGY

- ✓ Cordon the node
- ✓ Drain the node
- ✓ Fix underlying issue
- ✓ Uncordon and monitor
- ✓ Review alerts & logs

3 API SERVER UNAVAILABLE



SYMPTOMS

- kubectl commands timeout
- UI / dashboards unavailable
- Automation / CI/CD failing

ROOT CAUSES

- API server crash
- High load / memory
- Network / LB failure
- Cert / auth issues

INVESTIGATION COMMANDS

```
> curl -k https://<api-server>:6443/healthz
> systemctl status kube-apiserver
> journalctl -u kube-apiserver -f
> ps aux | grep apiserver
```

RECOVERY STRATEGY

- ✓ Check health endpoints
- ✓ Restart API server if needed
- ✓ Validate certificates
- ✓ Restore from known good config
- ✓ Monitor until stable

4 ETCD CORRUPTION



SYMPTOMS

- Cluster unstable
- Resources missing
- API errors / timeouts
- etcd alarms firing

ROOT CAUSES

- Disk corruption
- Power loss
- etcd bugs / version issues
- Full disk / IO errors

INVESTIGATION COMMANDS

```
> etcdctl endpoint health
> etcdctl alarm list
> etcdctl member list
> etcdctl --write-out=table endpoint status
```

RECOVERY STRATEGY

- ✓ Stop writes to etcd
- ✓ Restore from latest snapshot
- ✓ Recover cluster members
- ✓ Defragment etcd
- ✓ Verify data integrity

5 DNS FAILURES (COREDNS)



SYMPTOMS

- Pods can't resolve services
- External lookups failing
- Intermittent 503 / timeouts

ROOT CAUSES

- CoreDNS crash
- Config error in Corefile
- Upstream DNS issue
- Resource exhaustion

INVESTIGATION COMMANDS

```
> kubectl -n kube-system get pods -l k8s-app=kube-dns
> kubectl -n kube-system logs -l
> kubectl -n kube-system describe pod <coredns-pod>
> nslookup <service> <namespace>.svc.cluster.local
```

RECOVERY STRATEGY

- ✓ Restart CoreDNS pods
- ✓ Validate Corefile config
- ✓ Check upstream DNS
- ✓ Increase resources if needed
- ✓ Monitor DNS latency

6 INGRESS OUTAGES



SYMPTOMS

- Traffic returning 502/503
- Some apps unreachable
- High error rate at gateway

ROOT CAUSES

- Ingress controller crash
- Config / annotation error
- Cert / TLS issues
- Backend service down

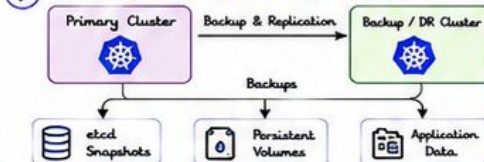
INVESTIGATION COMMANDS

```
> kubectl -n ingress-nginx get pods
> kubectl -n ingress-nginx logs -f
> kubectl get ingress
> kubectl describe ingress <ingress-name>
```

RECOVERY STRATEGY

- ✓ Restart ingress controller
- ✓ Validate ingress rules
- ✓ Check backend health
- ✓ Renew / fix certificates
- ✓ Test external traffic

7 DISASTER RECOVERY STRATEGY



KEY COMPONENTS

- Regular etcd snapshots
- Cluster configuration backup (Git)
- PV backups (Velero / CSI snapshots)
- Application level backups
- Documented runbooks

DISASTER RECOVERY FLOW

- 1 Detect major incident
- 2 Activate DR plan
- 3 Provision DR cluster
- 4 Restore etcd / workloads
- 5 Restore applications & data
- 6 Validate services
- 7 Communicate & monitor

BEST PRACTICES

- ✓ Test DR regularly
- ✓ Automate where possible
- ✓ Keep backups immutable
- ✓ RTO & RPO defined
- ✓ Post-incident review

COMMON INTERVIEW QUESTIONS



- Q1: How would you recover from a control plane outage?
- Q2: What steps do you take when etcd is corrupted?
- Q3: How do you debug DNS issues in Kubernetes?
- Q4: What commands do you use to troubleshoot node failures?

- Q5: How would you reduce ingress downtime?
- Q6: What is your recovery strategy for a full cluster loss?
- Q7: How do you ensure fast detection of incidents?
- Q8: How do you communicate during a major outage?



PRO TIP

Experience isn't about never failing. It's about recovering faster each time.



GOLDEN RULE

Stay Calm.
Follow the process.
Communicate.
Verify. Recover.

INTERVIEWER WANTS TO SEE:

- ✓ Real-world experience
- ✓ Structured thinking
- ✓ Problem solving
- ✓ Ownership & accountability



REMEMBER

Incidents happen. Resilience is built. Engineers who recover get trusted.

SENIOR KUBERNETES TROUBLESHOOTING INTERVIEW TOPICS

Senior interview questions. Real production troubleshooting.

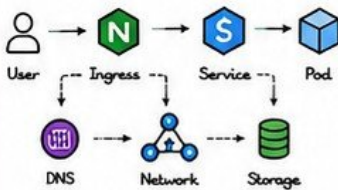
TOPICS COVERED:

- ✓ End-to-End Debugging
- ✓ kubectl Techniques
- ✓ Event Correlation
- ✓ Network Debugging
- ✓ Resource Analysis
- ✓ Root Cause Investigation

TROUBLESHOOTER MINDSET

- ★ Stay Calm
- ★ Gather Facts
- ★ Form Hypotheses
- ★ Test Methodically
- ★ Verify & Fix
- ★ Document & Learn

1 END-TO-END DEBUGGING



KEY POINTS

- Follow the request path.
- Test each component.
- Validate assumptions.
- Check both control plane and data plane.

PRO TIP

Always reproduce the issue before jumping to conclusions.

2 KUBECTL TECHNIQUES

ESSENTIAL COMMANDS

```

kubectl get all -A
kubectl describe <resource>
kubectl get events --sort-by=metadata.creationTimestamp
kubectl logs <pod> -c <container> --tail=200 -f
kubectl exec -it <pod> -- sh
kubectl top pods/nodes
kubectl explain <resource>
kubectl api-resources
kubectl get --raw /api
kubectl auth can-i <verb> <resource>
  
```

PRO TIP

Use -A to avoid missing resources in other namespaces.

3 EVENT ANALYSIS

KEY POINTS

- Events show what happened and why.
- Look for Warning and Failed events.
- Sort by timestamp.
- Events tell the story before logs.

COMMANDS

```

kubectl get events -A --sort-by=lastTimestamp
kubectl describe <resource>
kubectl get events --field-selector type=Warning
  
```

PRO TIP

Events are your first step in any investigation.

4 LOG CORRELATION

KEY POINTS

- Correlate logs across multiple components.
- Check timestamps.
- Use request IDs if available.
- Compare control plane and application logs.

TOOLS

```

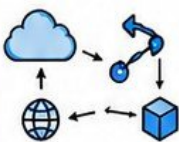
kubectl logs
kubectl logs --previous
kubectl logs -l app=myapp --all-containers
Centralized logging (EFK / Loki / ELK)
  
```

PRO TIP

Correlate by time window and request ID.

5 NETWORK DEBUGGING

CHECKLIST



- DNS resolution.
- Service reachability
- Pod-to-Pod connectivity
- Network Policies
- Ingress / LoadBalancer
- MTU / CNI issues
- mTLS / Certificates

COMMANDS / TOOLS

```

kubectl run netshoot --image=nicolaka/netshoot -it --rm
nslookup <service>
curl -v http://<service>:<port>
traceroute <pod-ip>
kubectl get networkpolicy -A
kubectl get endpoints <service> -o wide
  
```

PRO TIP

Use a debug pod (netshoot) inside the cluster to test connectivity like a pod.

6 RESOURCE ANALYSIS

KEY POINTS

- CPU / Memory / Disk pressure
- Quota and Limit issues
- OOMKilled / Evicted pods
- Node capacity and conditions
- Vertical / Horizontal scaling

COMMANDS

```

kubectl top pods/nodes
kubectl describe node <node>
kubectl get pods -A -o wide
kubectl get resourcequota -A
kubectl describe pod <pod>
  
```

PRO TIP

Check limits, requests and node allocatable. Resource issues cause most outages.

7 ROOT CAUSE INVESTIGATION

KEY PRINCIPLES

- Find the underlying cause, not just the symptom.
- Use the 5 Whys.
- Gather evidence.
- Document findings.
- Prevent recurrence.

INVESTIGATION FLOW

```

Collect Facts → Form Hypothesis → Test Hypothesis → Confirm Root Cause → Apply Fix → Verify → Document
  
```

PRO TIP

A good report today prevents an incident tomorrow.

SENIOR CHECKLIST

- ✓ Reproduce the issue
- ✓ Check recent changes
- ✓ Review Events
- ✓ Check Logs
- ✓ Validate Network
- ✓ Check Resources
- ✓ Review ConfigMaps / Secrets
- ✓ Check External Dependencies
- ✓ Form Hypothesis
- ✓ Test & Confirm
- ✓ Fix & Verify
- ✓ Document & Share

COMMON INTERVIEW QUESTIONS



- Q1: How would you troubleshoot a pod that is not starting?
 Q2: How do you investigate high latency in a Kubernetes app?
 Q3: How do you debug a Service that has no endpoints?
 Q4: How do you find which node a pod is running on and why?

- Q5: How do you troubleshoot DNS issues in Kubernetes?
 Q6: How do you debug network connectivity between pods?
 Q7: How do you find the root cause of an OOMKilled pod?
 Q8: How do you approach a Kubernetes production outage?



PRO TIP

The fastest fix is not always the best fix. Understand first.



GOLDEN RULE

Observe everything.
 Document everything.
 Learn everything.

INTERVIEWER WANTS TO SEE:

- ✓ Structured thinking
- ✓ Deep Kubernetes knowledge
- ✓ Practical troubleshooting skills
- ✓ Ownership & communication



REMEMBER

Great engineers don't guess. They investigate. Evidence → Analysis → Action → Improvement.

KUBERNETES & CLOUD NATIVE CHEAT SHEET

INTERVIEW TOPICS

Senior interview questions. Real production knowledge.

ENGINEER'S MINDSET

- ★ Automate Everything
- ★ Observe Everything
- ★ Secure Everything
- ★ Document Everything
- ★ Improve Everything

1 KUBECTL COMMANDS



CLUSTER INFO

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get namespaces
```

WORKLOADS

```
kubectl get pods -A
kubectl get deploy -A
kubectl describe pod <pod>
kubectl logs <pod> -f
kubectl exec -it <pod> -- /bin/bash
```

SERVICES & NETWORKING

```
kubectl get svc -A
kubectl get endpoints <svc>
kubectl get ingress -A
kubectl get networkpolicies -A
```

CONFIG & STORAGE

```
kubectl get configmaps -A
kubectl get secrets -A
kubectl get pvc -A
kubectl get pv -A
```

OTHER USEFUL

```
kubectl top nodes
kubectl top pods -A
kubectl get events --sort-by=.lastTimestamp
kubectl explain <resource>
```

💡 PRO TIP: Use -A for all namespaces.

2 HELM COMMANDS



REPO MANAGEMENT

```
helm repo add <name> <url>
helm repo list
helm repo update
```

SEARCH

```
helm search repo <chart>
helm show chart <chart>
helm show values <chart>
```

INSTALL / UPGRADE

```
helm install <release> <chart>
helm upgrade <release> <chart>
helm upgrade --install <release> <chart>
```

RELEASE MANAGEMENT

```
helm list -A
helm status <release>
helm rollback <release> <revision>
helm history <release>
helm uninstall <release>
```

TEMPLATING

```
helm template <release> <chart>
helm lint <chart>
```

💡 PRO TIP: Always version your releases.

3 ARGO CD COMMANDS



LOGIN

```
argocd login <server>
argocd logout
```

APPLICATIONS

```
argocd app list
argocd app get <app>
argocd app sync <app>
argocd app diff <app>
argocd app wait <app> --health
```

MANAGEMENT

```
argocd app create <app> --repo <url> \
--path <path> --dest-server <cluster> \
--dest-namespace <ns>
argocd app delete <app>
argocd app rollback <app> --revision <rev>
```

CLUSTERS

```
argocd cluster list
argocd cluster add <context>
```

PROJECTS

```
argocd proj list
argocd proj get <project>
```

💡 PRO TIP: Git is the source of truth.

4 DOCKER COMMANDS



IMAGES

```
docker images
docker pull <image>
docker rmi <image>
```

CONTAINERS

```
docker ps -a
docker run -d --name <name> -p 80:80 <image>
docker logs -f <name>
docker exec -it <name> /bin/sh
docker stop <name>
docker rm <name>
```

BUILD

```
docker build -t <image>:<tag> .
docker tag <image> <repo>/<image>:<tag>
docker push <repo>/<image>:<tag>
```

CLEANUP

```
docker system prune -a
docker volume prune
docker container prune
```

💡 PRO TIP: Keep images small.

5 CRICTL COMMANDS



RUNTIME INFO

```
crictl info
crictl version
crictl stats
```

CONTAINERS

```
crictl ps -a
crictl ps --name <name>
crictl inspect <container_id>
crictl logs <container_id>
crictl exec -it <container_id> -- /bin/sh
crictl stop <container_id>
crictl rm <container_id>
```

PODS

```
crictl pods -a
crictl pods --name <name>
crictl inspectp <pod_id>
```

💡 PRO TIP: crictl talks directly to the container runtime.

6 DEBUGGING WORKFLOW



1 IDENTIFY THE PROBLEM

What is failing?

```
kubectl get pods -A
kubectl get events --sort-by=.lastTimestamp
```

2 GATHER CONTEXT

Collect basic information.

```
kubectl describe pod <pod>
kubectl get all -n <ns>
```

3 CHECK LOGS

Application & system logs.

```
kubectl logs <pod> -f
kubectl logs <pod> -c <container>
```

4 CHECK RESOURCES

CPU, Memory, Storage.

```
kubectl top pods -n <ns>
kubectl describe node <node>
```

5 CHECK NETWORK

DNS, Services, Ingress.

```
kubectl get svc -n <ns>
kubectl exec -it <pod> -- nslookup <svc>
```

6 ISOLATE THE ISSUE

Reproduce or narrow down.

```
kubectl exec -it <pod> -- /bin/sh
Run curl, ping, tools inside pod
```

7 FIND ROOT CAUSE

Confirm & document.

Analyze findings → Hypothesize →
Prove → Fix → Verify

💡 PRO TIP: Always document what you find.

7 ARCHITECTURE CHECKLIST



- ✓ High Availability (Multi-AZ/Region)
- ✓ Control Plane, etcd, workloads resilient
- ✓ Scalability (HPA, Cluster Autoscaler)
- ✓ Observability (Logs, Metrics, Traces)
- ✓ Security (RBAC, Network Policies)
- ✓ Secrets Management
- ✓ Backup & Disaster Recovery
- ✓ Networking (CNI, DNS, Ingress, TLS)
- ✓ Cost Optimization
- ✓ Right-size nodes, autoscaling
- ✓ GitOps (Argo CD, Flux)
- ✓ Multi-tenancy (Namespaces, Quotas)
- ✓ Storage Classes & Data Management
- ✓ Compliance (Auditing, Policies)
- ✓ CI/CD Integration
- ✓ Documentation & Runbooks

💡 PRO TIP: A good design prevents incidents.

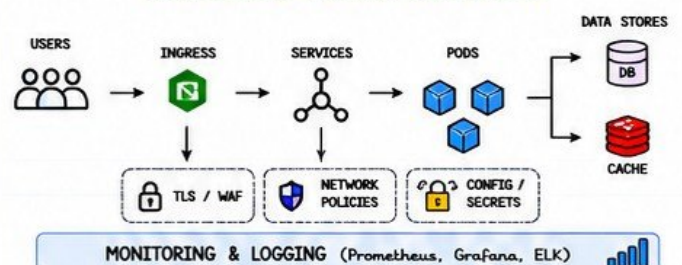
QUICK REFERENCE SHORTCUTS

- -n <ns> Specify namespace
- -A All namespaces
- -o wide Wide output
- -o yaml YAML output
- -l key=value Label selector
- --sort-by=.lastTimestamp Sort events by time
- kubectl drain <node> Drain node
- --ignore-daemonsets
- --delete-emptydir-data Remove emptyDir data
- kubectl cordon <node> Mark node unschedulable
- kubectl uncordon <node> Mark node schedulable

Remember

- ★ Observe.
- ★ Understand.
- ★ Hypothesize.
- ★ Test.
- ★ Fix.
- ★ Verify.
- ★ Document.
- ★ Improve.

ARCHITECTURE DIAGRAM (REFERENCE)



PRO TIP

The best engineers don't memorize commands, understand systems.



GOLDEN RULE

If you automate, document it.
If you document, version it.

INTERVIEWER WANTS TO SEE:

- ✓ Structured thinking
- ✓ Deep Kubernetes knowledge
- ✓ Practical troubleshooting skills
- ✓ Ownership & communication



REMEMBER

Incidents happen. Resilience is built. Engineers who recover get trusted.

CLOUD NATIVE ENGINEER FINAL ASSESSMENT

INTERVIEW TOPICS

Senior interview questions. Real production knowledge.

ENGINEER MINDSET

- ★ Think in Systems
- ★ Design for Scale
- ★ Automate Everything
- ★ Secure by Default
- ★ Run for Reliability
- ★ Never Stop Learning

TOPICS COVERED:

- ✓ Kubernetes Readiness
- ✓ Container Expertise
- ✓ GitOps Maturity
- ✓ Cloud Native Architecture
- ✓ Security Readiness
- ✓ Troubleshooting Assessment
- ✓ Production Engineer Scorecard

1 KUBERNETES READINESS



YOU SHOULD BE ABLE TO:

- ✓ Design and operate production clusters
- ✓ Deploy, scale and heal applications
- ✓ Troubleshoot complex cluster issues
- ✓ Secure workloads and control access
- ✓ Implement multi-tenant strategy
- ✓ Optimize resources and cost

ASSESSMENT SCENARIOS:

- ✓ A pod is CrashLooping -- how do you debug?
- ✓ Service has no endpoints -- what do you do?
- ✓ API server is unavailable -- next steps?
- ✓ Design a highly available Kubernetes cluster

2 CONTAINER EXPERTISE



YOU SHOULD BE ABLE TO:

- ✓ Write optimized Dockerfiles
- ✓ Build secure, minimal images
- ✓ Understand container runtime
- ✓ Troubleshoot container failures
- ✓ Manage images and registries
- ✓ Apply best practices

ASSESSMENT SCENARIOS:

- ✓ Why is a container exiting immediately?
- ✓ How do you reduce image size?
- ✓ Difference between Docker and containerd?
- ✓ How do you secure container images?

3 GITOPS MATURITY



YOU SHOULD BE ABLE TO:

- ✓ Explain GitOps principles
- ✓ Use Argo CD for deployments
- ✓ Implement declarative workflows
- ✓ Manage application lifecycle
- ✓ Troubleshoot sync failures
- ✓ Design GitOps at enterprise scale

ASSESSMENT SCENARIOS:

- ✓ App not syncing in Argo CD -- investigation?
- ✓ How do you handle config changes?
- ✓ GitOps vs traditional CI/CD?
- ✓ Multi-environment GitOps design?

4 CLOUD NATIVE ARCHITECTURE



YOU SHOULD BE ABLE TO:

- ✓ Design scalable cloud native systems
- ✓ Build microservices architecture
- ✓ Implement event-driven systems
- ✓ Choose the right architecture patterns
- ✓ Design for resilience and reliability
- ✓ Optimize for performance and cost

ASSESSMENT SCENARIOS:

- ✓ Design an e-commerce platform on Kubernetes.
- ✓ Implement blue-green deployment strategy.
- ✓ Build an event-driven order processing system.
- ✓ Design multi-region, highly available architecture.

5 SECURITY READINESS



YOU SHOULD BE ABLE TO:

- ✓ Implement Zero Trust principles
- ✓ Use RBAC and least privilege
- ✓ Secure containers and images
- ✓ Manage secrets and encryption
- ✓ Enforce network policies
- ✓ Pass security compliance audits

ASSESSMENT SCENARIOS:

- ✓ How do you secure a production cluster?
- ✓ Implement network policies for microservices.
- ✓ Best practices for secrets management?
- ✓ How do you prevent container breakout?

6 TROUBLESHOOTING ASSESSMENT



YOU SHOULD BE ABLE TO:

- ✓ Debug end-to-end issues quickly
- ✓ Use advanced kubectl techniques
- ✓ Analyze events and logs
- ✓ Correlate metrics and traces
- ✓ Find root cause, not just symptoms
- ✓ Lead incident response

ASSESSMENT SCENARIOS:

- ✓ Cluster is completely down -- recovery plan?
- ✓ DNS failures across the cluster -- how to fix?
- ✓ High CPU and memory -- investigation steps?
- ✓ Ingress returning 502/503 -- root cause?

7 PRODUCTION ENGINEER SCORECARD

SKILL AREA	WEIGHT	YOUR SCORE	MAX
Kubernetes Readiness	25%		100
Container Expertise	15%		100
GitOps Maturity	15%		100
Cloud Native Architecture	20%		100
Security Readiness	15%		100
Troubleshooting	10%		100
TOTAL SCORE	100%		600

READINESS LEVEL

- ★ 500 - 600 Principal/Staff (Expert)
- ★ 400 - 499 Senior (Production Ready)
- ★ 300 - 399 Mid-Level (Growing)
- ★ Below 300 Keep Learning (Focus Areas Needed)



GOLDEN RULES

- ✓ Understand the why, not just the how
- ✓ Always think in production
- ✓ Automate everything possible
- ✓ Design for failure
- ✓ Keep security at the center
- ✓ Continuously improve

WHAT'S NEXT?

